Abstract

This study focuses on the digital transformation of a traditional real estate agency
located in Juan Lacaze, Colonia, directed by Julio Perera Grasso, a professional with 21 years of experience
with experience in the sector. The problem addressed lies in manual management, based on
physical records and a weak digital presence, which generated operational inefficiencies and limited
the acquisition of new clients.

To overcome these deficiencies, an integrated system was designed and implemented, consisting of
two fundamental modules: an internal management platform and a corporate website.
The internal platform, developed with Java, Spring Boot and Java Swing, automates the
Registration and management of clients, properties, rental agreements, and payments. In addition,
It incorporates notification features for due dates and late payments, along with the
Automated generation of receipts and monthly reports for owners.
Additionally, the website, built using React, HTML, CSS, JavaScript and
Tailwind CSS offers an interactive catalog of properties that includes advanced filters.
to facilitate the search for the most suitable options, as well as a contact form
which allows establishing a direct communication channel.

The results obtained demonstrate a significant optimization of internal processes,
a substantial reduction in administrative errors and an improvement in the experience of both
customers as well as owners. In conclusion, the implementation of this digital system
It strengthens the competitiveness and expansion capacity of the real estate company, demonstrating that the
Technological modernization is crucial for sustainability in today's market.

Keywords

Customer

Team

Spring Boot

RESTful API

AWS (Amazon Web Services)

React

Frontend

Backend

MySQL

Agile development

Sprint

Scrum Methodology

Real Estate

Property Management

Properties for rent

User Interface (UI)

User experience (UX)

Index

1. Preliminary Project 7
1.1. Introduction.....7
1.2. Client Presentation 8
1.3. Problem Statement.....8
1.4. Alternatives and proposed solution.....10
1.5. List of needs 12
1.6. Project/Product Objectives.....13
1.7. Actors involved.....14
1.8. List of requirements.....15
1.8.1. Functional Requirements.....15
1.8.2. Non-functional requirements.....18
1.9. Scope and limitations.....19
1.10. Architecture 22
1.11. Design models and documents.....23
1.11.1. Diagram Ex1.....24
1.12. Project plan 24
1.12.1. Methodology / Framework.....24
1.12.2. Risk management plan.....25
1.12.3. Quality Plan (SQA).....30
1.12.4. Software Configuration Plan (SCM).....31
1.12.5. Training Plan.....32
1.12.6. Work schedule and effort estimate 34
1.12.7. Project Commitment 45
2. Project Development 46
2.1. Sprint 2 (Start date: 22/10/2024 end: 05/11/2024) 46
2.2. Sprint 3 (Start date: 06/11/2024 end: 20/11/2024) 54
23. Sprint 4 (Start date: 21/11/2024 end: 05/12/2024) 61
2.4. Sprint 5 (Start date: 06/12/2024 end: 20/12/2024).....75
2.5. Sprint 6 (Start date: 21/12/2024 end: 19/01/2025).....85
2.6. Sprint 7 (Start date: 20/01/2025 end: 03/02/2025).....99
2.7. Sprint 8 (Start date: 04/02/2025 end: 11/02/2025) 109

3. Project Summary .....112 3.1. Project Evolution .....112 3.1.1. Final Project Scope.....112 3.1.2. Deviations from the Preliminary Project .....118 3.1.3. Difficulties during Development 118
3.2. Project Management.....119 3.2.1. Working Methodology .....119 3.2.2. Management Tools Used.....120 3.3. Solution Design.....121
3.3.1. Final Architecture .....121 3.3.2. Use Case Diagram .....123 3.3.3. Class Diagram.....124 3.3.4. Final ER Diagram.....124
3.3.5. Technologies used.....125 3.4. Evidence of plan compliance .....126 3.4.1. SQA/Testing Plan .....126 3.4.2. SCM Plan 127
3.4.3. Training Plan.....128 4. Conclusions 130
4.1. Achievement of objectives.....130 4.2. Final analysis of the overall effort 131
4.3. Customer satisfaction 133
4.4. Lessons Learned.....134 4.5. Product Improvements .....135 4.6. Details for Accessing the Test Site 136
Glossary 137
Bibliography.....138
Annexes.....139
Appendix 1. Development of use cases.....139
Appendix 2. User Manual 166
Appendix 3. Deployment Manual .....166 Appendix 4. Testing Evidence.....170

1. Preliminary Project

1.1. Introduction

In this document you will find the process for developing the requirements of
project, the definition of work plans and an initial estimated timeline of the
times allocated for each task.

The project aims to develop a management system and a website for the
real estate agency of notary Julio Perera Grasso, located in Juan Lacaze, Colonia.
Currently, the real estate agency is managed manually, using index cards.
physical records for registering properties, clients, and contracts, which slows down the processes and
It limits the business's growth potential.

The proposed system will allow for the digitization of information and the automation of administrative tasks.
and improve operational efficiency. In addition, a website will be developed to facilitate the
Visibility of properties for sale or rent, allowing users to schedule visits
and conduct online consultations. This project seeks not only to optimize internal processes,
but also to increase the competitiveness of the business through a better digital presence.

The value of the project lies in its ability to modernize the management of the real estate company.
reducing administrative times and improving the customer experience, which translates into
greater efficiency and growth potential for the company.

1.2. Client presentation

Our client's name is Julio Perera Grasso, he is originally from the city of Juan Lacaze,
Colonia, works as a Notary and Lawyer, but his main business is a
real estate company which he has owned for 21 years, as a sole proprietorship.
It stands out in the city for offering a wide variety of rental properties.
The company consists of him (owner) and his secretary (employee).

The real estate agency has a Facebook page (which is rarely used) where it
They display the various properties available for rent, but the main way to
Renting is the direct attendance of potential clients at the studio inquiring about
houses for rent, either because the real estate agency is well-known in the city or because you see a sign
Advertising is available at the various properties we have for rent. Clients can...
You can contact the office by phone, via WhatsApp to the real estate agency's number, or you can
appear there in person to inquire about a specific property or
to find out what type of properties they have for rent.

1.3. Problem Statement

Currently, their business focuses on the real estate sector, which is managed as follows:
manner:

-A client arrives at the office to consult with the secretary at the counter.
to inquire about rental properties or about a specific property, which you
They reported it via Facebook or a "for rent" sign available on the property.

-Once the price has been agreed upon and if the client is satisfied with the amenities of the house, the secretary will...
Ask the customer what type of guarantee they have (Anda, Porto, Contaduría, Sura, owner of
a house that can be used as collateral or LUC).

-A contract is signed, which has a date, an owner, a tenant, a price and a term (1
year, 2 years, etc.) and certain rights and obligations.

-Once the contract is signed, the house is handed over to the tenant.

-The tenant is asked to change the names on the UTE and OSE receipts to their name.

-As of the current date (October 2024), if the rental price is less than 20,590 pesos, the
The secretary initiates a procedure through the DGI website requesting exemption from IRPF, for the purposes
to avoid that tax for the owner, a procedure that is done every 2 years of the contract.

-Rent payment can be made in cash at the counter (it is the least common payment method)
common) or by bank transfer to the real estate agency's account.

-The real estate agency charges a commission of 10%+VAT on the rental price and issues an invoice
(They have electronic billing). The homeowner is then paid the following amount:
rental amount minus said commission.

The system for tracking rentals by the real estate agency is not currently used.
No software of any kind. All this information is handled with physical records, per client.
which becomes very tedious, because the secretary has to search in the client index
There are owners in the office to find the one for a specific client. There are even
clients who own more than one rental property, which makes the search more complex.

In turn, our client would like to have a more formal medium than a simple webpage
Facebook to showcase properties to its clients and offer through this medium a
A direct line of contact with them to coordinate visits to the different houses. Also
Consider using advertising services on other social media platforms to reach a wider audience.
people, especially people from other cities.

1.4. Alternatives and proposed solution

With the aim of digitizing customer records and streamlining the search process
These services are offered to the client to develop a Java system for installation on computers.
from the office. This system will provide the option to register owner clients (including
details such as full name, telephone number, bank account and how you are
pays) and will assign a specific directory to each one. Within each client's directory
The data of the various properties that he made available to the
real estate agency for rentals. A "property stock" will also be displayed.
It will indicate how many of that client's properties are rented and how many are vacant.
Information that will be stored about these properties will include their address, contracts,
rent, whether such rents are exempt from income tax or not, and the way in which the tenant pays the
rent (cash at the office counter or by bank transfer) and will be shown
if the UTE and OSE services for the current month were paid (otherwise, the following will be displayed)
number of months of delay).

The system will also have a section displaying a calendar, in which
Our client will be able to upload important dates such as rental expiration dates, among others.
others, and the system will take care of showing you automatic on-screen notifications when
These dates are approaching.

Also, based on the stored information, the system will send automatic notifications.
to the administrator when a rent payment is overdue, when the services of a
property has not been paid for or when after a time determined by the
The administration in the system configuration has not changed the names of the receipts
OSE and UTE in the name of the tenant of a property.

When a rental payment is made, the system will provide the option to issue a receipt.
payment after filling in some fields with information about it (which month it is paid,
Address of the property being rented, name and ID number of the tenant, amount paid, cost
of the administration, VAT of the administration, etc.).

In addition, the system will be able to issue a monthly settlement report to be delivered to the client owner with all the important information that he/she requires (total amount to collect in the month, which of your properties were rented, etc.)

With the aim of offering our client a more formal means than a webpage Facebook, to showcase rental properties to its clients, we decided to develop a website Website for the real estate agency using HTML, CSS, JavaScript and the React framework.

The site will have a homepage with an image of the real estate agency and below it will have a A brief introductory text will be provided. It will also include a "Services" tab where... There will be illustrative images of the services they offer and a detailed description of the same.

The site will also feature a properties tab where they will be presented in the form of catalog of rental properties with photos and a brief description below each one. Clicking on a property will redirect you to a page for that property, where the following will be displayed: More photos will be available, zooming in will be allowed, and detailed features will be displayed. estate.

The properties will be loaded centrally into the system's database. management, the site will access them and update automatically when they are added or Properties are removed. This makes the maintenance work much more practical. The site has been updated because we no longer need to register our existing properties again. the management system or constantly checking that there are no uploaded properties that They are currently unavailable.

On each property page there will be a "Search" button that, when pressed, will display a A form where you enter your phone number, email address, and message. Below there will be a The "Send" button, when pressed, sends an email with that message to the recipient's email address. real estate agency, which will be received by the secretary who will contact the client to address your concerns and questions (for example, to coordinate a visit to the property).

The site will also feature a contact tab detailing the various ways to get in touch. to contact the real estate agency (email, WhatsApp number, landline).

At the bottom of the website there will be a footer containing hyperlinks to social networks

social media accounts of the real estate company and will invite clients to follow them.

And last but not least, we will provide our client with advice on

help him create a company Instagram account, hire advertising services

Instagram and create an advertising reel for the real estate agency in order to attract potential clients

clients through this social network. When registering a new property in the system, this

It will offer the option of automatically publishing said property on the website and social networks of

automatically from a single location, greatly speeding up the task.

1.5. List of needs

-Digitize customer records and their rentals.

-Have it organized and receive notifications of rental deadlines, payment defaults and

name changes for utility services (OSE and UTE) of the properties, expiration dates of

procedures for exemption from personal income tax for each office rental contract (each one has

its own expiration date, given that it has its own signature date).

-To offer a more formal means than a Facebook page to showcase properties to the

clients and coordinate visits to them.

-Open an Instagram account for the real estate agency, upload the same content that is uploaded to the Facebook page.

This, but also uploading some stories looking for interactions.

-Pay for Instagram advertising to attract potential customers.

-Have a registered directory for each client and within each one have all the

information about the houses that the real estate agency has available.

-When entering a client's directory, have at a glance the data on how many houses that client owns.
how many are rented and how many are available.

-Ability to store contracts in each client's directories.

-When selecting a house, you can see the payment status of the services for that house (if it was paid).
light, water, etc.).

-When selecting a house, you can see if the rental is exempt from income tax or not.

-When selecting a house, you can see if the rent for that month was paid and, if not,
show how many days late the payment is.

-When registering a customer of type owner in the system, being able to register their number of
telephone number, property details, your bank account (optional) to which payment should be made
deposit the rental amounts if not paid in cash, your ID and the
the way in which the customer is paid.

-Being able to issue a rental receipt in accordance with the information in each contract (date,
tenant's name, payment month, home address, amount paid.

-To be able to issue a settlement report to the client owner detailing how much
The rent was charged, the amount paid (if applicable) to DGI for IRPF,
administration fee, administration VAT fee, and balance due for the
owner.

1.6. Project/Product Objectives

Based on the list of identified needs, we have defined the following objectives:
what we aim to achieve upon completion of the project:

ÿ Create a professional website to showcase properties and coordinate viewings, which will professionalize the image of the real estate agency.

ÿ Facilitate access to and management of information through the digitization of records clients and rentals.

ÿ Establish an alert system for the expiration of rental deadlines, payments and procedures, which will help prevent problems and improve operational efficiency.

ÿ Implement a directory that centralizes all information about clients and their properties, facilitating consultation and management.

ÿ Generate detailed settlement reports and rent receipts, which will allow a Clearer and more organized financial management for owners.

1.7. Actors involved

The beneficiaries of this project will be the members of the real estate company, because that the management system will provide them with facilities that will streamline their task. Customer owners will also benefit from the implementation of project because the system will allow them to receive a settlement report detailed and concise monthly statement with all costs and relevant information about your properties that are being rented.

We, as the development team, will be responsible for implementing and to run both the management system and the real estate website.

The person in charge of making the project decisions will be our client, Julio Perera. Grasso, owner of the real estate agency.

The real estate company will be responsible for financing the project, including advertising expenses. on social networks, system servers, web domains, among other things.

1.8. List of requirements

1.8.1. Functional Requirements

Functional requirements for a real estate management system in JAVA:

• **RF1. Login:** The system must provide an authentication screen where only the administrators can log in with the username and password registered by They previously. Dependencies: None. Priority: MANDATORY.

• **RF2. Registration of owner clients:** The system must allow the registration of homeowner clients and assign a directory to each one. In turn, it must store relevant data of the customer owner (first and last name, address, phone number, ID number, payment method, bank account).
You must register at least one property to become a client of the real estate agency. (The same data is stored as in RF3). Dependencies: RF1. Priority: MANDATORY.

• **RF3. Owner Customer Management:** Allows you to manage the information of the owner clients. And from there perform the following functionalities:

or **RF3.1 View owner card:** A list of clients is displayed and

When you select one (you can search by name and/or ID number) the following is displayed

Owner's information sheet that includes:

ÿ **Personal data** ÿ **List**

of registered properties

ÿ **Registered rental agreements**

ÿ **View list of rental receipts**

ÿ **See list of monthly settlements.**

Dependencies: RF1, RF2. Priority: MANDATORY.

or **RF3.2 Add property to an owning client:** The system must allow the record of the rental properties that each client owns. At least

They must register the following data (Registration Number, Address, House/Apartment, Photos,

Common expenses (if applicable). Dependencies: RF1, RF2.
Priority: MANDATORY.

or **RF3.3 Deleting a customer's property:** The system must allow
To remove from the rental agreement a property that the client will no longer rent. Dependencies: RF1, RF2. Priority: MANDATORY.

or **RF3.4: Contract Registration:** The system must allow the registration of contracts rental properties and link them in the database to the client who owns the property house. It also indicates whether or not it exempts from income tax according to the amount of the Rental. Dependencies: RF1, RF2 and RF3. Priority: MANDATORY.

or **RF3.5: Rental tracking:** The system must allow data storage relevant information about a client's rental properties (if the services or not, if the tenant is up to date with rent payments, register a service payment, I see or update the contract expiration date). Dependencies: RF1, RF2, RF3, and RF4. Priority: MANDATORY.

- **RF4: Expiration notifications:** The system must provide the possibility to the real estate managers enter important dates into a calendar (expiration of rental periods, expiry of personal income tax exemption procedures) and he must issue notifications when these dates approach. Dependencies: RF1, RF2, RF3, RF4 and RF5. Priority: MANDATORY.
- **RF5: Notifications of late payments and changes of service names:** The The system must issue notifications to the real estate administrators when In a house, the rent has not been paid (in addition to how many days late it is), the services or when the tenant has not changed the utility bills to their name at a time determined by the administration in the configuration of System. Dependencies: RF1, RF2, RF3, RF4 and RF5. Priority: MANDATORY.
- **RF6: Issuance of rental receipt:** The system must have a section for select a rental agreement stored in the system, select a month To pay, select whether the Ministry of Economy index applies or not (if applicable, enable). a field to enter it and let the system handle the calculation) and when you press a button to issue a rental receipt with the date, tenant's name, amount that

Payment and details of the house being rented. Dependencies: RF1, RF2, RF3, RF4 and RF5. Priority: DESIRABLE.

- **RF7: Issuance of monthly rent settlement report:** The system must have a section to issue a monthly settlement report to the client owner detailing the amount charged to the tenant, the amount of IRPF that the owner had to pay to DGI in that month (if applicable), the administration fee, administration VAT and total balance to be collected said month. Dependencies: RF1, RF2, RF3, RF4 and RF5. Priority: DESIRABLE.
- **RF8: Customer Property Stock:** When selecting a customer, the system should display on screen how many properties are occupied and how many are free. Dependencies: RF1, RF2, RF3, RF4 and RF5 Priority: DESIRABLE.

Functional requirements for the real estate website:

- **RF9: Website Homepage:** The website must have a homepage with a photo of the real estate agency and a brief description. Facilities: None. Priority: DESIRABLE.
- **RF10: Rental Property Catalog:** The website must have a tab for properties in which rental properties are presented in a catalog format with photos and a brief description below each one. Offices: None. Priority: DESIRABLE.
- **RF11: Property search engine:** The website must have a property search engine properties that allow filtering by different criteria (price range, location of the property, number of rooms, whether it has a patio or not, whether it has a garage or not, if it has (Barbecue or not, whether it has a pool or not, etc.). Dependencies: RF12. Priority: DESIRABLE.
- **RF12: Details of each property:** When clicking on a property, the site should redirect to a page on this site, where more photos will be displayed, it will be possible to do Zoom in on them to see detailed features of the property. Outbuildings: RF12. Priority: DESIRABLE.
- **RF13: Property Inquiry:** Place a button on each property page

“Consultar” which, when pressed, displays a form in which a number is entered a phone number, an email, and a message. Below there will be a "Send" button that... Pressing it sends an email with that message to the real estate agency's email address. which will be received by the secretary who will contact the client to address their concerns and questions (for example, to coordinate a visit to the Property). Dependencies: RF13 and RF14. Priority: DESIRABLE.

- **RF14: Services tab:** The site must have a services tab where you can Show illustrative photos, text describing the different services offered by the Real estate and its working methods. Offices: None. Priority: DESIRABLE.
- **RF15: Contact tab:** The site must have a contact tab where you can Please provide details of the different ways to contact the real estate agency (email, WhatsApp number, landline number). Dependencies: None. Priority: DESIRABLE.
- **RF16: Footer:** At the bottom of the page there will be a footer that will contain hyperlinks to the real estate agency's social media and will invite clients to Follow them. Dependencies: None. Priority: DESIRABLE.
- **RF17: Online Chat:** The site must have an online chat on the right side of the that customers will be able to communicate in real time with the secretary of the Real estate. Dependencies: None. Priority: DESIRABLE.
- **RF18: Simultaneous publication of properties:** The system must to have a section where, when registering a new property, it is possible to do so through a button to publish it on the real estate agency's website, on the Facebook page and on the Instagram account. Dependencies: RF1, RF2 and RF3. Priority: DESIRABLE.

1.8.2. Non-functional requirements

- **RNF1: Service language:** The service must be translated into Spanish. Dependencies: X Priority: MANDATORY.
- **RNF2: Interface:** The service must have a user-friendly interface Dependencies: X Priority: DESIRABLE.

- **RNF3. Data Security:** The client's personal data, their property and Contracts must be securely encrypted. Type: Regulatory. Dependencies: X Priority: MANDATORY.

- **RNF4: Security in data modifications and deletions:** Access to Modifications and deletions of sensitive data will be carried out once completed Authentication Dependency: X Priority: REQUIRED.

- **RNF5: Web:** The web application will be compatible with the main browsers (Chrome, Microsoft Edge and Firefox) Dependencies: X Priority: REQUIRED.

- **RNF6: Errors:** The system must display error messages that are descriptive For the end user Dependencies: X Priority: DESIRABLE.

- **RNF7: Creating an Instagram account:** Create an Instagram account of the Real estate. Offices: None. Priority: OPTIONAL.

- **RNF8: Hiring Instagram advertising service:** Pay for the service Instagram's advertising platform helps posts reach more people. Dependencies: RNF7. Priority: OPTIONAL.
- **RNF9: Creating an advertising reel:** Create an eye-catching reel that explains Briefly describe the services the real estate agency offers to appear in the Instagram ads. Dependencies: RNF7, RNF8. Priority: OPTIONAL.
- **RNF10: Cloud data backups:** The system must perform them automatically Back up your data and store it in the cloud. Priority: MANDATORY.

1.9. Scope and limitations

The project aims to develop a management system in Java that facilitates internal administration of a real estate agency and offer key functionalities for your site This web system will allow you to register and manage client owners, including data. Personal documents, assigned properties, rental agreements, and payment receipts. In addition,

They will be able to register and deregister properties, view the stock of available properties, and Generate monthly settlement reports.

A crucial part of the system will be the issuance of automatic notifications, alerting about contract expirations, payment delays, and other significant events related to the properties. For clients, the system will integrate a website with a catalog of rental properties, a search engine with various filters, and a form to submit consultations and scheduling visits. There will also be a feature to publish automatically properties on social networks, such as Facebook and Instagram, and to Generate rental receipts automatically, including tax calculations.

The project also includes the creation of a user-friendly interface, compatibility with major browsers and a high level of security in handling of the data. However, because the development team consists of only Two people, mandatory functionalities will be prioritized to ensure delivery within of the deadline, which is February 11. Optional features, such as the creation of Instagram accounts or social media advertising contracts are only They will implement it if time allows.

The project also includes the implementation of an online chat on the website of the real estate agency. This functionality will allow clients to communicate in real time with the real estate agency secretary to answer questions, coordinate visits and receive information Regarding the properties. This characteristic is considered a desirable requirement. Therefore, its implementation will depend on the availability of time during the project development. However, in case of setbacks or unforeseen events, the Chat functionality could be postponed to focus on mandatory requirements that ensure the functional core of the system.

In summary, the project is focused on meeting the essential requirements for To ensure the efficient operation of the system. It is expected that, upon delivery of the product Finally, the customer's expectations are met, while maintaining quality and a focus on the agreed deadlines. Optional features will be adjusted according to the evolution of the project and the possible setbacks that may arise during development.

Required:

- **RF1. Login.**
- **RF2. Register of owner customers.**
- **RF3.1 Display owner card.**
- **RF3.2 Add property to a customer owner.**
- **RF3.3 Deregister a customer's property.**
- **RF3.4: Contract registration.**
- **RF3.5: Rental tracking.**
- **RF4: Expiration notifications.**
- **RF5: Notifications of late payments and change of service name.**
- **RNF1: Service language.**
- **RNF3. Data Security.**
- **RNF4: Security in data modifications and deletions.**
- **RNF5: Web.**
- **RNF10: Cloud data backups.**

Desirable:

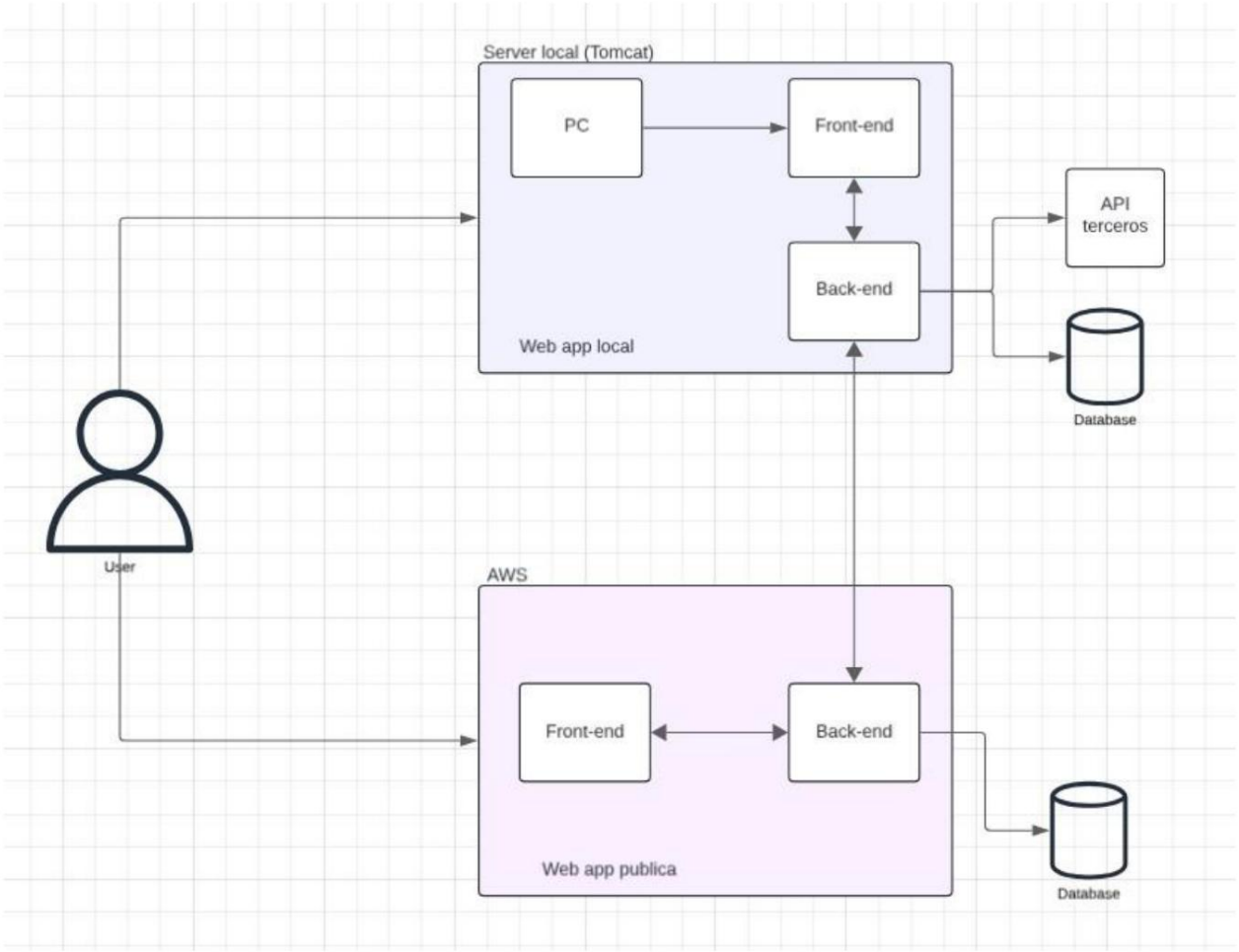
- **RF6: Issuance of rental receipt.**
- **RF7: Issuance of monthly rent settlement report.**
- **RF8: Stock of client properties.**
- **RF9: Website homepage.**
- **RF10: Catalog of properties for rent.**
- **RF11: Property search engine.**
- **RF12: Details of each property.**
- **RF13: Property query.**
- **RF14: Services tab.**
- **RF15: Contact tab.**
- **RF16: Footer.**
- **RF17: Online chat.**
- **RF18: Simultaneous publication of properties.**
- **RNF2: Interface.**

- **RNF6: Errors.**

Optional:

- **RNF7: Creating an Instagram account.**
- **RNF8: Hiring Instagram advertising service.**
- **RNF9: Creation of advertising reel.**

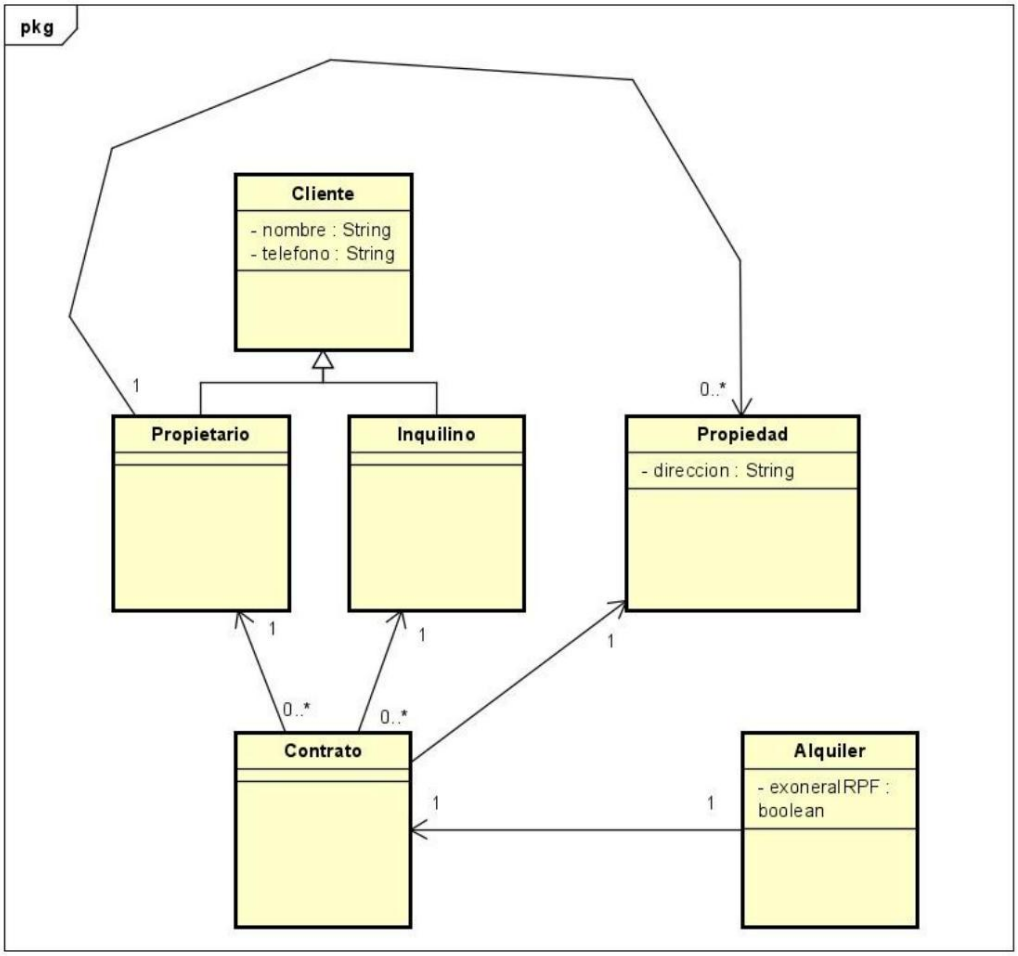
1.10. Architecture



The architecture is divided in two, since we will have two different types of users.
First, we have the real estate administrator user, who will have a website
local app, having communication with the Front-end, Back-end and database via the PC
of data, all of this locally, while also adding the third-party API. This will help
to the creation and removal of posts on Facebook and Instagram. On the other hand, we have
with the public Web app, the same one used by the real estate agency's clients for management
of the properties, view property listings and more; the website will be hosted on AWS.

1.11. Design models and documents

1.11.1. Diagram Ex1



1.12. Project plan

1.12.1. Methodology / Framework

For the implementation of the project to develop the management system for the real estate company in Java will be used, employing an adaptation of Scrum, an agile framework that facilitates incremental delivery. of value. Scrum has a set of good practices and artifacts (example: division into sprints, retrospective and replanning meetings, etc.) that we understand fit the type project, allowing for the effective management of complexity, promoting the Flexibility in the face of change and continuous improvement.

The sprints, or short work cycles, will generally last 2 weeks each, in which the They will develop functional enhancements to the system.

Throughout each sprint, the team will participate in daily meetings (Daily Scrum), where Each member will report on their progress, plans for the day, and any impediments. that could affect the work. These meetings allow for fluid communication and ensure that the team is aligned in their daily tasks.

At the end of each sprint, a sprint review will be conducted, in which the following will be presented: increase developed to the stakeholders, who will provide feedback on the product. This evaluation will allow the work plan to be adjusted if necessary. In addition, will conduct a sprint retrospective, in which the team will review the process of development, will identify areas for improvement and adjust its methods to optimize work in the following sprints, fostering continuous improvement in the project.

1.12.2. Risk management plan

ID	Detail	Probability Impact	Level of exposure

R1	Delivery delays due to changes in requirements	Half	High	6
R2	Customer's lack of familiarity with the use of the system	High	Half	6
R3	Security problems in the management of sensitive data	Half	High	6
R4	System instability due to data load	Half	High	6
R5	Dependence on external providers (API) that after	High	Half	6
	updates cause incompatibility			
R6	Poor estimation of the time needed to perform each task	High	High	9

R7	Lack of availability of one of the team members	Half	High	6
R8	Technological failures (possible breakdowns in work equipment)	Half	High	6

Table 1. Identified Risks

Identification Mitigation Plan		**Contingency plan**
R1	Clearly define the requirements from the beginning. Use Scrum to iterate quickly and adapt to small changes in each sprint, minimizing the impact on delivery times.	Establish a timeline with room for minor changes. Prioritize features and negotiate with the client which changes are essential and which can be postponed to a future version.
R2	Include customer training and a user manual. Also, design the system with a user interface. intuitive and user-friendly to minimize complexity.	Provide ongoing technical support for a period after the launch.

R3	Implement encryption in the storage and transmission of the data. Conduct audits of periodic security checks and guarantee only authorized users access the information. Make regular backups of your data.	Having a rapid response protocol in In the event of a data breach, this includes measures such as changing credentials, restoring from secure backups, and performing emergency audits.
R4	Optimize database queries and design. Monitor the performance regularly.	Prepare a scalability plan that includes additional servers or hardware upgrades if demand grows significantly.
R5	Implement notification systems that alert when an external API is failing or not responding correctly. Define alternative procedures in case of prolonged unavailability.	Store critical information locally in case of prolonged API outages. Automate connection retries when APIs are down. restore.

R6	Re-evaluate initial estimates For each sprint, based on the progress of the previous sprint, break down large tasks into smaller subtasks to improve estimation accuracy, implement time buffers in the schedule to cover potential deviations, and hold daily meetings to adjust task scope if deviations are detected.	In the event that a poor estimate is detected that affects the project schedule, mandatory and desirable requirements will be prioritized together with the client, eliminating or postponing optional ones to ensure on-time delivery.
R7	When planning, consider the possibility that one of the project developers may be unavailable. If one of the developers is unavailable, the other must commit to taking on additional work hours until their colleague returns. Include time buffers in the schedule for these situations. cases.	
R8	Having backups stored on GitHub and also having backup computers in case the one being used for development breaks down.	Download a backup copy of the project to one of the backup computers and continue with the workflow.

Table 2. Risk Management

1.12.3. Quality Plan (SQA)

Documentation

To ensure compliance with the University's requirements, we will rely on the following throughout the project documentation in the format provided in document 302.

Testing

At the end of each sprint, testing will be performed through a series of tests to verify the
To ensure its proper functioning, the aim is to make sure that all functionalities
are validated during the course of the project.

To improve product quality and ensure the testing process in a way
We will fully integrate black-box and white-box testing. With the black-box process
We will be able to ensure that the functional requirements meet the requirements and are
The inputs and outputs of the system produce the expected results; all of this
We will be able to view it from the end user's perspective. On the other hand, with the process
With white box testing, we can optimize software performance while simultaneously identifying
logical errors, thus ensuring quality and time reduction from early stages
and costs.

Development

Standards and best practices established based on our experience will be followed.
selecting those we deem appropriate, this to ensure that the code is clean,
easy to maintain and scalable.

ÿ To name variables or methods, start with lowercase letters and use Camel.
Case.

ÿ The code will be commented to facilitate understanding at specific times and
necessary.

ÿ Design patterns such as expert and MVC will be used.

1.12.4. Software Configuration Plan (SCM)

Version management:

Version control for the project will be handled with Git using GitHub for ease of use
the collaboration of both members of the development team.

For the collaboration between the two developers, the Copy-
ModifyMerge. Each developer will work on a local copy of the source code. Upon completion
In one task, the code will be merged with the main branch, handling any potential conflicts.
efficiently.

The workflow will be divided into 4 branches: Main (or Master), Develop, Feature and Hotfix.

Main (or Master): Stable branch with production-ready versions.

Develop: Branch where functionalities are integrated before being promoted to production.

Feature: Each person will work on their own feature branch for each assigned task.
Example: feature/RF1-Login, feature/RF2-RegistroClientes

Hotfix: In case of critical errors in production, a hotfix branch can be created.
to correct errors and then merge to Main.

Task distribution:

Developer 1: Will be in charge of the implementation of the backend (Java, Springboot).

Developer 2: Will be in charge of the frontend (HTML, CSS, React and JavaScript) and the integration of the real estate website.

Both will be able to collaborate on critical functionalities such as authentication, security, and database.

Merge frequency:

From Feature to Develop: A merge to Develop will be performed when a feature is completed. after passing the code review and the corresponding unit tests.

From Develop to Main: This will be done at the end of each sprint, when both parties have tested and integrated its various functionalities.

Hotfix to Main: If a critical bug occurs in production, a hotfix branch will be created to Correct the problem immediately. Once resolved, a direct merge will be performed to Main and then to Develop to maintain consistency across both branches.

1.12.5. Training Plan

1.12.5.1. Development team training

As part of the preparation for the project, we require training in new
Technologies such as reinforcing the knowledge previously acquired during the degree. Each
The sprint has defined the time we will use to carry out the training.

The following technologies will be learned through courses (from YouTube and Udemy) and the documentation of these:

- React
- Springboot
- Java
- Git

1.12.5.2. Customer training

To ensure effective customer training, we will implement two activities
complementary. First, we will hold face-to-face sessions in which
We will demonstrate the use of the system and its different functionalities. During these sessions,
We will be in constant communication with the client, gathering feedback at the end of each
sprint, which will allow them to acquire prior and contextual knowledge about the tool.

In addition, we will provide a series of demonstration videos that address each of the
system functionalities. These resources will be available for the client to use.
consult at any time, thus ensuring a comprehensive and effective understanding of
all the features offered.

1.12.6. Work schedule and effort estimate

- **Sprint 1 (Preliminary Project)** - Start date: 03/09/2024 end: 21/10/2024 (121 hours)

Task	**Estimated hours**
Meetings with the client	12
Meetings with the tutor	10
Training in new technologies	12
Preliminary project documentation	51
Workshop on work methodology and preliminary project structure	36

- **Sprint 2** - Start date: 22/10/2024 End date: 05/11/2024 (92 hours)

Task	**Estimated hours**
Environment configuration	8

Initial database design/ Class diagram	4
Meetings with the client	4
Tutoring meetings	4
Sprint documentation	4
Training/Research	10
Sprint testing (integration)	4
Buffer for unforeseen events	10
RF1: Login	10
RF2: Owner Customer Registration	24
RF3.1 View data sheet Partial owner: Include only the client's details and a list of the properties they have for rent, leaving the interface ready to add one	6

new property at a visual level.	
Sprint deployment	4
Total estimated hours	92

- **Sprint 3 - Start date:** 06/11/2024 End date: 20/11/2024 (107 hours)

Task	**Estimated hours**
Meetings with the client 4	
Tutoring meetings	4
Sprint 4 Documentation	
Training/Research	10
Sprint testing (integration)	4

Contingency buffer 10	
RF3.2: Add property to an owning client	14
RF3.3: Deregister a customer's property.	10
owner RF3.1 record: (partial): The list of registered contracts is added, with the possibility of adding a new contract from that interface.	8
RF18: Publication of simultaneous properties	35
Sprint deployment	4
Total hours dear	107

- **Sprint 4** - Start date: 21/11/2024 End date: 05/12/2024 (94 hours)

Task	**Estimated hours**
Meetings with the client	4
Tutoring meetings	4
Sprint 4 Documentation	
Training/Research 10	
Sprint testing (integration)	4
Buffer for unforeseen events	10
RF4: Expiration notifications	20
RF5: Notifications of late payments and name changes services	20
RF6: Issuance of rental receipts	14

Sprint deployment	4
Total estimated hours	94

- **Sprint 5** - Start date: 06/12/2024 End date: 20/12/2024 (102 hours)

Task	**Estimated hours**
Meetings with the client	4
Tutoring meetings	4
Sprint 4 Documentation	
Training/Research 10	
Sprint/ integration testing	4
Buffer for unforeseen events	10
RF7: Issuance of monthly rent settlement report	16

RF8: Stock of client properties	8
RF9: Website homepage	12
RF10: Rental Property Catalog	24
Sprint deployment	4
Total estimated hours	100

- **Sprint 6** - Start date: 21/12/2024 End date: 19/01/2025 (102 hours)

Task	**Hours dear**
Meetings with the client	4

Meetings of tutorial	4

Documentation of the sprint	4
Training/Research	10
Sprint/ integration testing n	4
Buffer for unforeseen events	10
RF14: Tab services	10
RF15: Contact tab	12
RF16: Footer	2
RF13: Property Inquiry	10

RF17: Online Chat	24
RF12: Details of each property	4
Sprint deployment	4
Total estimated hours	102

- **Sprint 7 -** Start date: 20/01/2025 End date: 03/02/2025 (94 hours)

Task	**Hours dear**
Meetings with the client	4

Meetings of tutorial	4

Documentation of the sprint	4
Training/Research	10
Buffer for unforeseen events	10
RF3.4: Contract Registration	14
RF3.5: Rental tracking	10
RF11: Property Finder	18
Final testing	16
Sprint deployment	4
Total estimated hours	94

- **Sprint 8** - Start date: 04/02/2025 End date: 11/02/2025 (105 hours)

Task	**Estimated hours**
Meetings with the client	4
Meetings of tutorial	4
Documentation of the sprint	12
Buffer for unforeseen events	10
UX/UI settings to level of web application	15
Correction bugs	20
Documentation project end	36

Sprint deployment	4
Total estimated hours	105

1.12.7. Project Commitment

The development team commits to dedicating 50 hours per week to the project (25 each one) already comply with the requirements specified within the deadlines agreed with the client.

For their part, the client agrees to maintain constant communication with the team Development support via WhatsApp to answer questions, as long as it is between 9 am and 5 pm. hs. In turn, it also commits to holding virtual meetings at a time to be coordinated with end of each sprint. In case our client is unavailable, the secretary

The real estate company representative will be responsible for answering the development team's questions and assisting them. videoconferences.

2. Project Development

2.1. Sprint 2 (Start date: 22/10/2024 end: 05/11/2024)

PLANNED TASKS:

- Environment configuration.
- Initial database design/ Class diagram.
- RF1: Login.
- RF2: Registration of owner customers.
- RF3.1 Display owner record (partial): Include only the customer data and a List of properties you have for rent, leaving the interface ready to add. a new property at a visual level.
- Sprint testing (integration).
- Sprint documentation.

Requirements prioritization (MoSCoW)

To manage the planning and replanning of tasks in this sprint, it has been applied the MoSCoW methodology to define priorities and ensure the achievement of the most critical objectives.

Must Have (Essential)

- RF1 (login): is essential to allow access to the system, establishing a secure authentication mechanism.

Should Have (Important but not critical)

• RF2 (registration of owner customers): essential for customer management,
But in case of setbacks, it could be adjusted or finalized in a sprint.
later.

Could Have (Desirable if time permits)

• RF3.1 (view owner record (partial)): not essential for the
It does not affect the main function, but its inclusion improves the user experience.

Environment configuration:

At the beginning of the sprint we started by configuring the environment, installing IntelliJ IDEA, creating a Spring Boot project and uploading it to a Git repository so that the two members of the development team could work on simultaneous comfortably.

Later, when developing the graphical interfaces of the application that in IntelliJ is implemented using JavaFX, and we encounter the problem of There are very few tutorials in Spanish about this technology on the internet, so which made us change our IDE used by Apache NetBeans and develop the graphical interfaces with the Java Swing library, which we had already worked on previously in the race.

Initial database design/ Class diagram

During this phase, we focused on creating the initial design of the base of data and the class diagram, as we consider appropriate based on the needs and requirements set by the client. Based on the Based on the conversations and details provided, we structured a proposal that It seeks to efficiently reflect the key relationships and entities for the correct system operation.

Meetings with the client:

We had a brief meeting with the client to clarify some details and resolve
doubts about the requirements. Thanks to this meeting, we were able to obtain a
better understanding, which allowed us a better comprehension of the design of the
database and class diagram.
He also told us about RF2, which we had agreed to in the preliminary project.
that in order for a client to be registered in the system, at least one registration was required
property to the same, that in some cases he wanted to be able to store clients by
more so than those who at that time do not have properties managed by the real estate agency,
because he wants to have their phone numbers stored in case he needs them.
accidentally delete your contact on your cell phone.
In turn, in the preliminary draft we had agreed to store the form in which it was
He pays the owner upon registration, but later the client clarified
which has clients who own more than one rental property, some of which
The owner is paid in cash and others by bank transfer, therefore,
It was decided that this data would be stored when each rental is registered in
specific.

Training/Research:

During the sprint, a significant amount of time was dedicated to performing
Technical training to strengthen the team's skills in two technologies
key: JavaSwing and Spring Boot.

RF1: Login:

We configured a login controller with an initial endpoint that returned
“Login successful” only if the username admin and password admin were entered

(without connecting to a database yet), otherwise, it would return “Username or password incorrect.”

We performed various tests in Postman by executing requests to this endpoint and They were successful.

We then set up a MySQL database and connected it to our Spring Boot project using the spring-boot-starter-data-jpa library. We create a table “user” with one attribute for the username and another attribute for the password. We inserted a record with username “admin” and password “admin” and We repeated the tests in Postman, which were successful.

RF2: Registration of owner customers:

We set about configuring the client controller to handle requests of type POST, GET, DELETE, PUT and PATCH (even without connecting to a database, but using hardcoded data).

Later, when we connect our Spring Boot project to the database, We created a "client" table with the following attributes: id, name, surname and address.

RF3.1 View owner record (partial):

A window was created to view the record of an owner received by parameter (for this first hardcoded version) in which the attributes of this (name, surname, address and a list of properties).

Sprint testing (integration):

The Apache project was successfully connected to a MySQL database built in MySQL Workbench.

Sprint deployment:

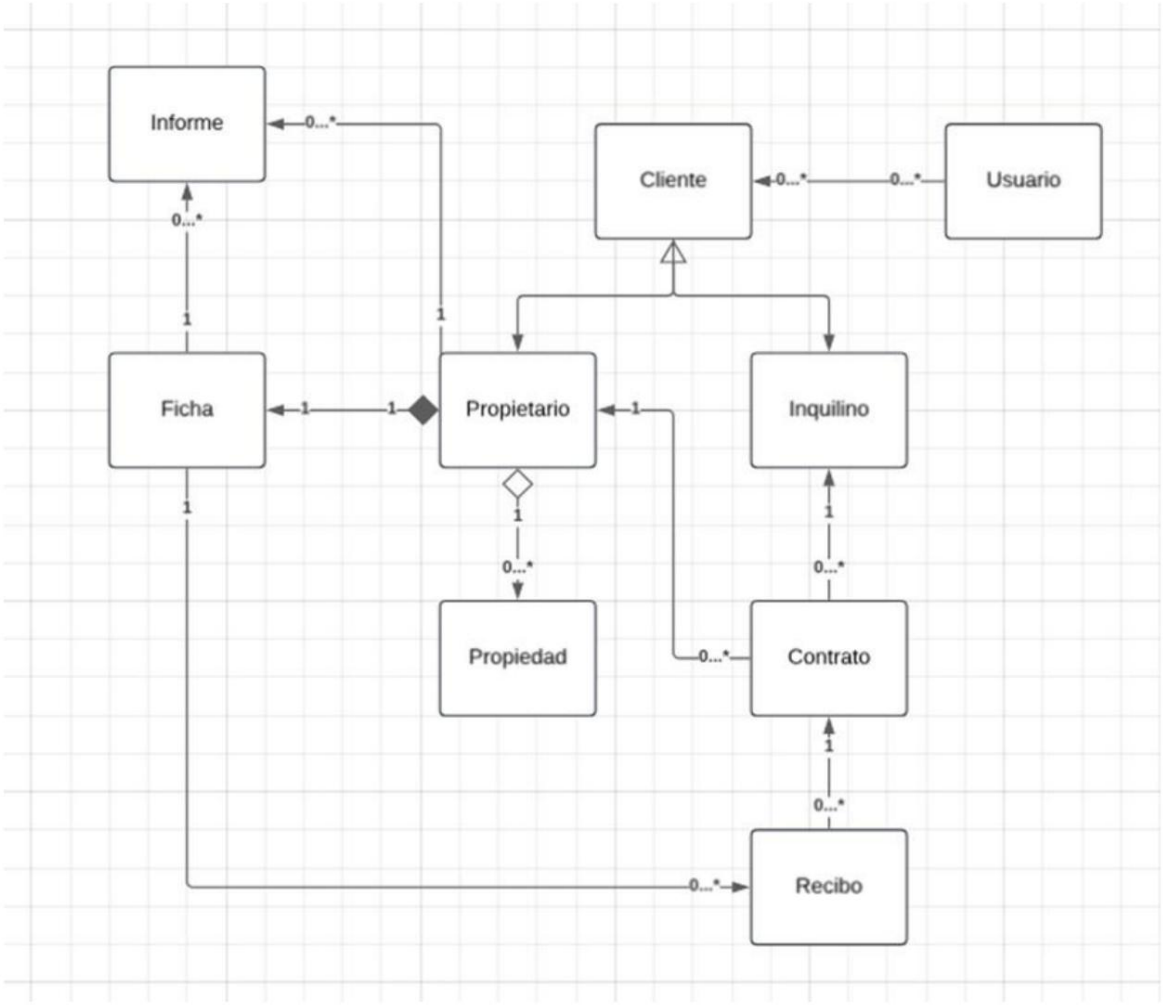
In this sprint, although the deployment had initially been planned, it was decided postpone it until the next one, as we believe we would be better off.
trained to carry it out after the scheduled workshop.

The only thing that wasn't fully implemented was RF3.1, which consisted of
View the owner's records where the list of contracts is added
registered, because we had decided to make this requirement
We've only partially designed the viewing window; we still need to finalize the functionality via the API (this requirement will be finalized in the next sprint)

LIST OF UNPLANNED TASKS

- Changing the environment from IntelliJ using JavaFX to Apache using Java Swing.
- Review of Java Swing knowledge.

PRELIMINARY DESIGN DOCUMENTS:



SPRINT TESTING:

The development of the use cases that were tested in this sprint (CU001, CU002 and CU003) They are found in Annex 1 and the evidence of the testing is found in Annex 4.

MANAGEMENT TASKS/MEETINGS:

In the sprint kickoff meeting with our tutor, we agreed that the first thing we would do

It would involve creating a Spring Boot project and implementing a simple login without connecting to a database.

data still (just implement the endpoint to make a post request and that

validate the credentials passed as a parameter).

RISK MONITORING:

No risks were presented in this sprint, neither those indicated in the preliminary project nor new ones.

STRESS ANALYSIS:

Sprint 2 – (Deliverable 1) - Start date: 22/10/2024 End date: 05/11/2024

Tarea	Tiempo estimado	Tiempo real
Configuración del ambiente	8	8,5
Diseño inicial de base de datos/ Diagrama de clases	4	13
Reuniones con el cliente	4	1
Reuniones de tutoria	4	4
Documentación del sprint	4	6
Capacitación/Investigación	10	19
Testing del sprint (integración)	4	1
Buffer para imprevistos	10	5
RF1: Login	10	11,5
RF2: Registro de clientes propietarios	24	15
RF3.1 Visualizar ficha de propietario (parcial)	6	1,5
Deploy del sprint	4	0
Total horas	92	85,5

The objective of this sprint was to successfully implement a system of User authentication, develop the new customer registration process owners and create a partial view for displaying the owner's card.
All of this had to work in an integrated way with the API and the database, ensuring a smooth and efficient experience.

CONCLUSIONS AND REPLANNING OF THE SPRINT STACK

At the conclusion of this sprint, we can confirm that we were very close to the estimates. Initially, in terms of hours dedicated. Although we faced some difficulties in Starting with the development of the graphical interface using JavaFX, we learned to adapt efficiently and migrate to Java Swing without this affecting significantly the estimated time nor will it generate delays in other requirements. Despite this setback, we managed to complete all the objectives set out with success.

Looking ahead to the next sprint (Sprint 3), we have decided to carry out a small This modification is based on the experience gained during this cycle. With this improvement, We seek to optimize the process and ensure that we can repeat the success achieved. in this sprint.

Tarea	Horas estimadas
Reuniones con el cliente	4
Reuniones de tutoría	4
Documentación del sprint	4
Capacitación/Investigación	10
Testing del sprint (integración)	4
Buffer para imprevistos	10
RF3.2: Agregar propiedad a un cliente propietario	14
RF3.3: Dar de baja propiedad de un cliente.	10
RF3.1: Ficha del propietario (parcial): Se agrega la lista contratos registrados y la posibilidad de agregar un nuevo contrato desde esa interfaz.	8
Deploy del sprint	4
Total de horas estimadas	72

The change made consisted of postponing the implementation of RF18 (publication (of properties simultaneously) for the next sprint, due to the above previously.

2.2. Sprint 3 (Start date: 06/11/2024 end: 20/11/2024)

PLANNED TASKS:

• RF3.2: Add property to an owning client

• RF3.3: Deactivate a customer's property. •

RF3.1: Owner record (partial): The list of registered contracts and the possibility of adding a new contract from that interface.

• Training/Research

• Sprint deployment •

Sprint testing (integration)

Requirements prioritization (MoSCoW)

To manage the planning and replanning of tasks in this sprint, it has been applied the MoSCoW methodology to define priorities and ensure the achievement of the most critical objectives.

Must Have (Essential)

• RF3.1 (owner's form (partial)): the functionality must be completed

View the list of registered contracts and allow the creation of a new one

You can sign the contract from this interface, as it is key to managing rentals.

Should Have (Important but not critical)

• RF3.2 (add property to an owning client): is a feature

relevant, but if there are unforeseen circumstances, it could be rescheduled without affecting the main function of the system.

Could Have (Desirable if time permits)

- RF3.3 (terminate customer ownership): not essential in this sprint, but its Implementation would improve property management.

Training/Research:

For this sprint we had to research how to upload files from the Java Swing interface to store contracts in the database such as PDF files.

Meetings with the client:

In the meeting we had with our client, we reminded him of what he had asked us for. that it should appear on the customer's record and told us that regarding that function, no I didn't want the list of receipts for each client to appear, nor the list of settlements. monthly because the window would become "too cluttered" and he prefers to look for them directly in the folder where he always stores these types of files.

RF3.2: Adding a property to an owning client:

A list of properties was added to each client's record window and It added an "Add Property" button that opens a registration form for a New property. When the registration of the new property is confirmed, it is displayed A message saying that the registration was successful and using a callback It automatically updates the client's property list in their window file.

RF3.3: Deleting a customer's property:

A "Delete Property" button was added below the property listing in the
window of each client's record that when a property is selected from the list and
Pressing it displays a confirmation window, and in case the administrator
Confirm that if you want to remove the property, the frontend communicates with the API.
using a DELETE request and removes the selected property. In case
If there is an error, it displays the corresponding message.

RF3.1: Owner's record (partial): The list of registered contracts is added and the possibility of adding a new contract from that interface:

The "contract" table was created in the database. A list of contracts was added to it.
the window for each client's record, in addition to an "Add contract" button that
Open a registration form for a new contract. When the registration is confirmed
From the new contract, a message is displayed saying that the registration was successful and
The contract list is automatically updated using a callback.
of the client in their profile window.

Sprint deployment:

The sprint deployment was successfully completed using AWS with the concepts
learned in the deployment workshop.

Sprint testing (integration):

All implemented functionalities were tested and results were obtained
They were successful in these tasks; there were no integration problems.

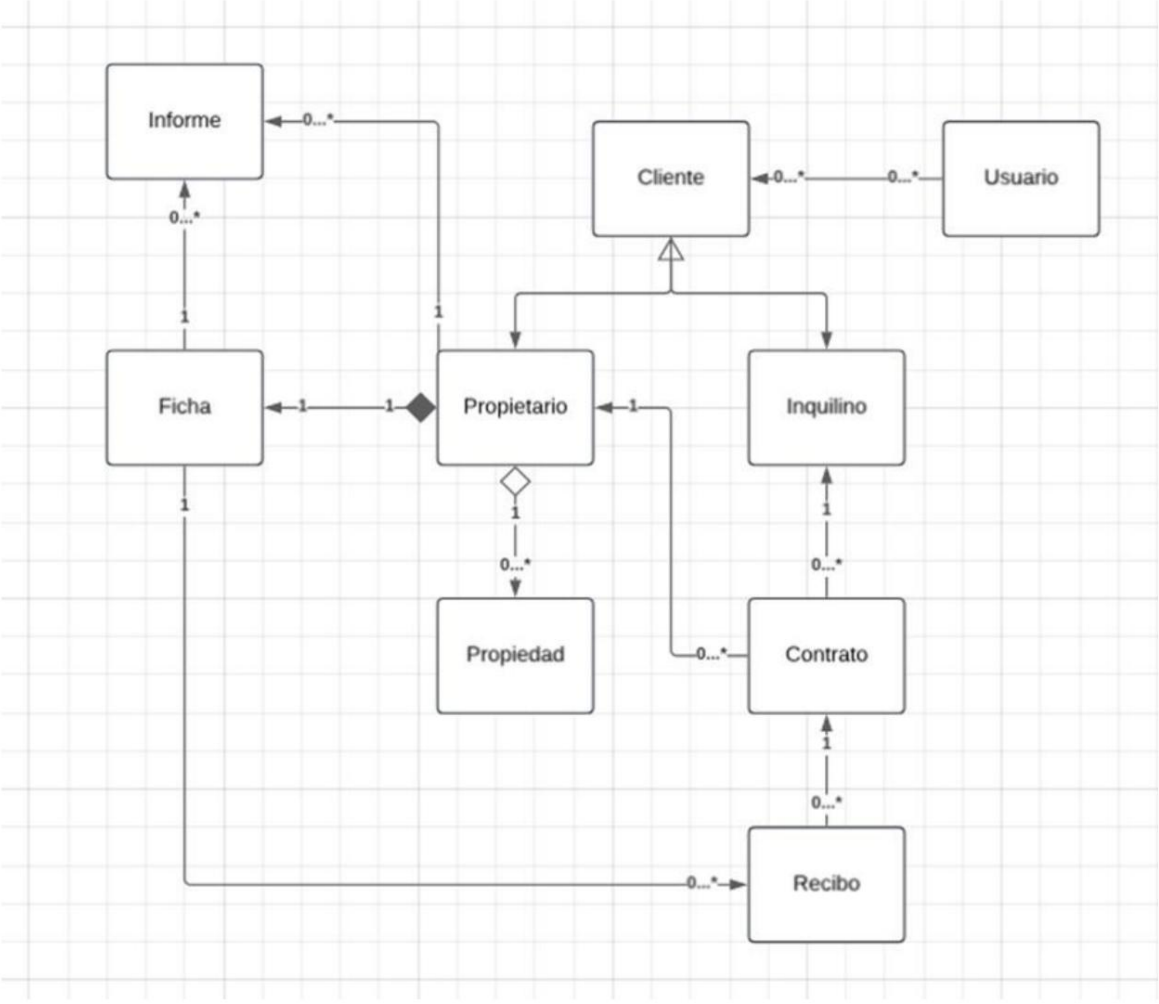
LIST OF UNPLANNED TASKS

-Division of the project into 2 separate parts (the frontend on one side and the backend on the other) and adaptation of logic in general.

RISK MONITORING

No risks were encountered in this sprint, neither those indicated in the preliminary project nor new.

PRELIMINARY DESIGN DOCUMENTS



SPRINT TESTING:

The development of the use cases that were tested in this sprint (CU004, CU005 and CU006) are found in Annex 1 and the evidence of testing is found in Annex 4.

MANAGEMENT TASKS/MEETINGS

In the first meeting we had with our tutor halfway through the sprint, we told him regarding the delays we experienced due to the project being divided into 2 and the adaptations that we had to make. In turn, he recommended that we attend the deployment workshop and We used AWS for our case, as it was the service that best suited our needs. project.

During the meeting with our client, the new windows were shown to him and he was invited to try them out. to perform different records. He was satisfied with our progress and asked us to we added new attributes to the properties that he considers necessary for when later be published on the website.

STRESS ANALYSIS

Tarea	Tiempo estimado	Tiempo real
Reuniones con el cliente	4	2
Reuniones de tutoría	4	1
Documentación del sprint	4	3
Capacitación/Investigación	10	8
Testing del sprint (integración)	4	4
Buffer para imprevistos	10	10
RF3.2: Agregar propiedad a un cliente propietario	14	12
RF3.3: Dar de baja propiedad de un cliente.	10	5
RF3.1: Ficha del propietario (parcial): Se agrega la lista	8	10
contratos registrados y la posibilidad de agregar un nuevo contrato desde esa interfaz.		
Deploy del sprint	4	8
Total horas	72	63

The goal of this sprint was to add new properties to existing clients directly from the interface, as well as to remove existing properties. Additionally, the sprint aimed to implement the functionality to upload contracts to these clients through the interface. Finally, the objective was to deploy the functionality upon completion.

CONCLUSIONS AND REPLANNING OF THE SPRINT STACK

At the conclusion of this sprint we can affirm that we were very close to the estimates

Initially, in terms of the hours dedicated. The only major difficulty we encountered

It was to divide the project into 2 separate ones (one for the back-end and one for the front-end), already

We had to move many things and adapt others to make them work as before.

That's how we used up the 10 hours of our contingency buffer.

Something we'll keep in mind for future sprints is to hold a formal meeting

with the client before starting the project, clearly explaining the functionalities

to implement and tell us clearly what needs to be present in each one, in order to provide you with a solution that best suits your needs and to avoid rework by developers due to not being entirely clear on the requirements to be implemented.

Our client explained that they need more stored information about the stored properties, therefore, for the next sprint we decided to add a A new task focused on the same thing, since it is something essential that we need to complete. before carrying out RF18. In turn, we decided to postpone the RF4 and RF5 requirements for the next sprint, otherwise we won't be able to complete the others tasks within the stipulated time.

Then sprint 4 would be organized as follows:

Tarea	Horas estimadas
Reuniones con el cliente	4
Reuniones de tutoría	4
Documentación del sprint	4
Capacitación/Investigación	10
Testing del sprint (integración)	4
RF3.2: Agregar propiedad a un cliente propietario (agregar atributos faltantes)	14
RF6: Emisión de recibos de alquiler	14
RF18: Publicación de propiedades en simultáneo	35
Deploy del sprint	4
Total de horas estimadas	93

23. Sprint 4 (Start date: 21/11/2024 end: 05/12/2024)

PLANNED TASKS:

- Training/Research
- Sprint testing (integration)
- RF3.2: Add property to an owning client (add missing attributes)
- RF6: Issuance of rental receipts
- RF18: Simultaneous property listing
- Contingency buffer
- Sprint deployment

Requirements prioritization (MoSCoW)

To manage the planning and replanning of tasks in this sprint, it has been applied the MoSCoW methodology to define priorities and ensure the achievement of the most critical objectives.

Must Have (Essential)

- RF6 (issuance of rental receipts): is fundamental for financial control of the system, as it directly impacts payment management and the transparency with the owners.

Should Have (Important but not critical)

- RF3.2 (add property to a client owner (add missing attributes)): It's a significant improvement for property management, but if any issues arise Unforeseen circumstances could be rescheduled without affecting critical operations.

Won't Have (Will not be included in this sprint)

• RF18 (concurrent property publishing): In this sprint, we will attempt to Sync posts with both Facebook and the website. Without However, if a viable solution for automatic renewal is not found of the token in Facebook Developers, integration with said platform and publication will be limited solely to the website.

Training/Research:

During the training process on how to connect to Facebook for posting Regarding properties, we conducted research on how, through the application of From the desktop, you can create a post that will be published simultaneously to the feed of Facebook. To do this, we investigated the use of Facebook Developers and its integration with their platform. API.

We also investigated how to improve the application's graphical interface using Java Swing and how to generate PDF receipts from the application using the iTextPDF library.

Contingency buffer:

We use the hours allocated to the contingency buffer to make improvements to the interface. application graphics, in order to provide a better visual presentation to our client and that the interface is user-friendly.

RF3.2: Add property to an owning client (add missing attributes):

More attributes were added to the Property class based on discussions with our client. the sprint kickoff meeting, and in this way the logic of this was completed so that in

In the next sprint, we can focus on publishing these properties on the page.

The attributes of the Property class are as follows:

-Id

-Address

-Description

-Number of bathrooms

-Number of rooms

-Number of bedrooms

-Number of garages

-Neighborhood

-City

-Number of plants

-Accepts pets (boolean)

-Guy

-Total square meters

-Square meters built

-Square meters of land

-Current UTE Owner

-Current OSE holder

-Type of roof

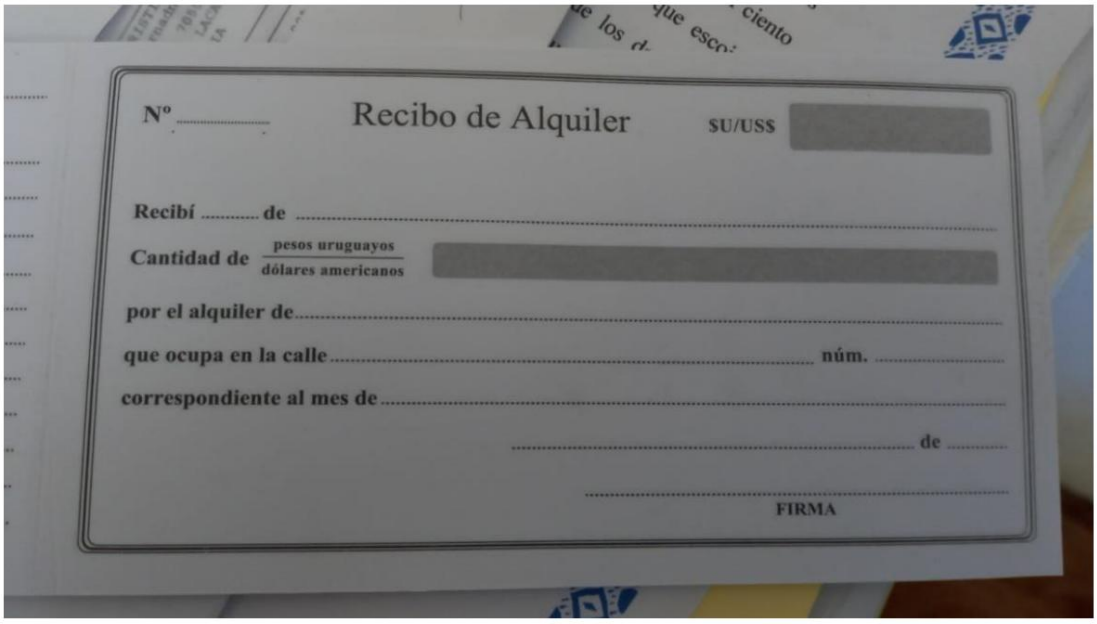
-Owner

-Photos

RF6: Issuance of rental receipts:

For issuing rental receipts, we rely on a sample receipt that our

A customer provided us with the forms, which they currently fill by hand and deliver to their customers:



In the functionality of the owner client profile, we added a section that allows

View the list of rentals from that owner, and when you select one and press the button

Clicking “Generate report” displays a form that allows you to generate a PDF report by filling it out.

the “Month paid” field and press the “Generate PDF” button. To carry out this

For functionality, we use the iTextPDF library in its version 7.1.16.

RF18: Simultaneous property listing:

By using Facebook Developers and its API integration, we identified several

problems both on the platform and in the customer experience. In particular,

We discovered that, to make simultaneous posts to the Facebook feed, it was

It was necessary to renew the tokens periodically, which had to be done manually every 30

days. Since this renewal cannot be automated and was a desirable requirement,

We decided not to proceed with its implementation so as not to create more work or affect the customer experience.

Because of this, we decided to limit the requirement for property listings in simultaneous only to the website, which we will develop during the next sprint.

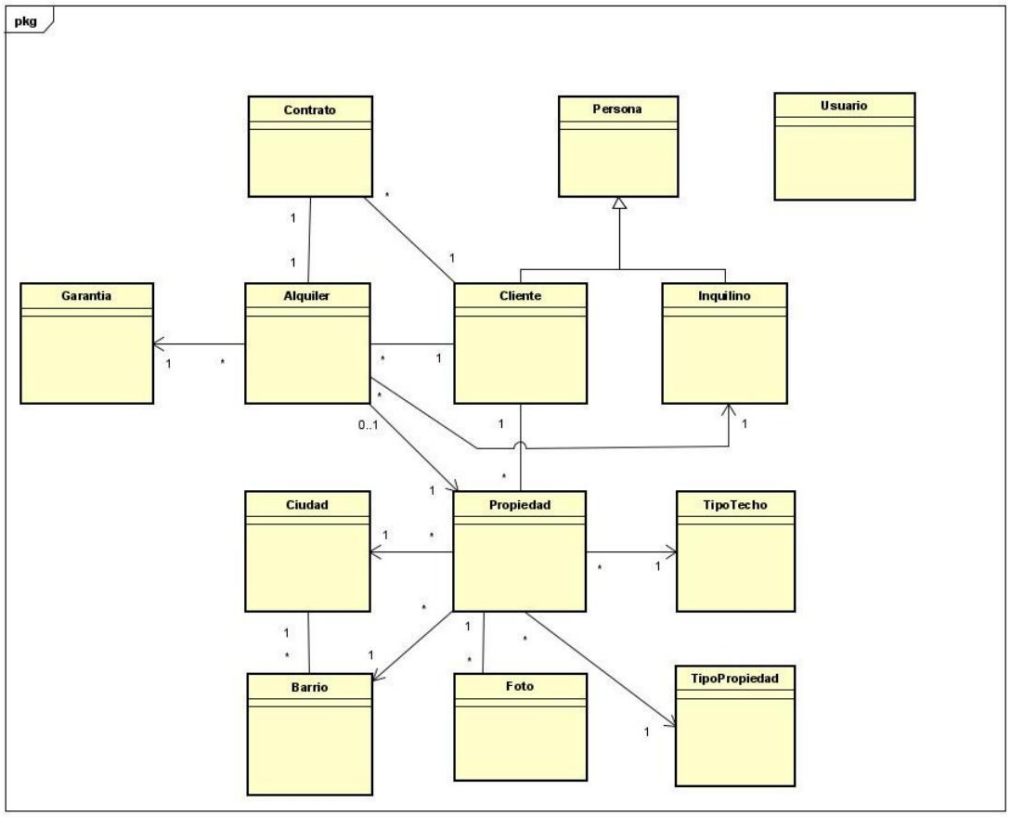
Sprint deployment:

We decided not to deploy in this sprint because we focused on trying to Complete the requirement for simultaneous property publication and make improvements in the application's graphical interface to begin thinking of the application as a product Finally, make it as complete as possible, both in terms of functionality and appearance. visual.

LIST OF UNPLANNED TASKS:

-Improvements to the menu and visual presentation of the application.

PRELIMINARY DESIGN DOCUMENTS:



SPRINT TESTING:

The development of the use cases that were tested in this sprint (CU007 and CU008)

They are found in Annex 1 and the evidence of the testing is found in Annex 4.

MANAGEMENT TASKS/MEETINGS:

In the meeting we had with our client at the start of the sprint, a receipt was requested.

Example rental to use as a template for issuing PDF receipts in the system and

He was asked in detail what information about the properties he needed to save

Yes, definitely, both in the internal system and on the website. He clarified that there is data that is not

are mandatory (such as total square meters, built square meters and meters

square meters of land, since sometimes they do not have this data).

In the meeting we had with our tutor halfway through the sprint, we showed him the progress. that we made to the functionality of adding a property to a client and the improvements that We did to the application's graphical interface.

STRESS ANALYSIS:

Task	Estimated time	Real time
Meetings with the client 4		4
Tutoring meetings	4	2
Sprint 4 Documentation		4
Training/Research 10		22.5
Testing of the sprint (integration)	4	4
Buffer for unforeseen events	10	17
RF3.2: Add property to a client owner (add attributes) missing)	14	16

RF6: Issuance of receipts rent	14	16
RF18: Publication of simultaneous properties	35	25
Sprint deployment	4	0
Total hours	103	110.5

The goal of this sprint was to finalize the functionality of the property registration.
Adding the missing attributes will enable the functionality to issue rental receipts.
Implement simultaneous property listings on Facebook, Instagram, and the website
web and finally deploy the sprint.

RISK MONITORING:

The following risks were identified during this sprint:

- R6: Poor estimation of the time required to perform each task. This risk affects us.
It was presented within the time we estimated for training for this sprint; we had allocated 10
hours but we ended up dedicating 22.5 and several hours from the contingency buffer due to everything
We had to do a lot of research on YouTube and internet forums to issue receipts

PDF from the system, make improvements to the graphical interface using Swing and the training we had to do on how to use Facebook Developers to connect with Facebook's API for simultaneous posting when registering a new property.

We were forced to implement the contingency plan we had prepared in the preliminary plan in case this risk arose, which was to prioritize the requirements mandatory and desirable, giving higher priority to mandatory ones. After all the training we did on how to connect to the Facebook API and saw that it wasn't working to be feasible for the project to implement the functionality of simultaneously publishing the properties on that social network, we decided to disregard this requirement and adjust our subsequent sprint stack prioritizing mandatory requirements, which in their Most of these are functionalities of the internal system, which our client needs in greater extent.

CONCLUSIONS AND REPLANNING OF THE SPRINT STACK:

Taking stock of this sprint, we can conclude that we successfully implemented It successfully met the RF6 and RF3.2 requirements, but we decided to make some modifications to the RF18 requirement. Due to the complications presented by the Facebook platform Developers decided that the publication of properties will only be done in the the real estate agency's website, not Facebook and Instagram as we had planned. We had planned in the preliminary project, in case of complications with the Requirements classified as desirable/optional may be modified or discarded in order to emphasize the requirements that our client considers mandatory for the application. RF18 was listed as “DESIRABLE”.

Thinking about the next sprints, taking into account the experience gained in the

Given the previous options and the fact that the delivery date is getting closer, we decided to rule them out because

The following requirements are currently being considered, with the aim of focusing on those that are indeed relevant.

required according to our client:

- RF17: Online Chat: The site must have an online chat on the right side of the
  that customers will be able to communicate in real time with the secretary of the
  Real estate. Dependencies: None. Priority: DESIRABLE.
- RNF7: Creating an Instagram account: Create an Instagram account of the
  Real estate. Offices: None. Priority: OPTIONAL.

- RNF8: Hiring Instagram advertising service: Pay for the service
  Instagram's advertising platform helps posts reach more people.
  Dependencies: RNF7. Priority: OPTIONAL.

- RNF9: Creating an advertising reel: Create an eye-catching reel that explains
  Briefly describe the services the real estate agency offers to appear in the
  Instagram ads. Dependencies: RNF7, RNF8. Priority: OPTIONAL.

Taking into account the changes mentioned above, our sprint stack of

From now on it would look like this:

- **Sprint 5 - Start date: 06/12/2024 End date: 20/12/2024 (104 hours)**

Task	**Estimated hours**
Meetings with the client	4

Tutoring meetings	4
Sprint documentation	4
Training/Research	10
Sprint testing	4
Contingency buffer	10
RF7: Issuance of monthly rent settlement report	12
RF9: Website homepage	8
RF18: Simultaneous publication of properties on the website	20
RF10: Rental Property Catalog	24
Sprint deployment	4
Total estimated hours	104

- **Sprint 6 - Start date: 21/12/2024 end: 19/01/2025 (99 hours)**

Task	**Estimated hours**
Meetings with the client	4
Tutoring meetings	4
Sprint documentation	4
Training/Research	10
Sprint testing	4
Contingency buffer	10
RF8: Stock of client properties	8
RF4: Expiration notifications	20
RF5: Notifications of late payments and changes of service names	16
RF14: Services tab	8

RF15: Contact tab	6
RF16: Footer	1
Sprint deployment	4
Total estimated hours	99

- **Sprint 7 - Start date: 20/01/2025 end: 03/02/2025 (100 hours)**

Task	**Estimated hours**
Meetings with the client	4
Tutoring meetings	4

Sprint documentation	4
Training/Research	10
Buffer for unforeseen events	10
Contract Registry	6
RF3.5: Rental tracking	10
RF11: Property Finder	18
Final testing	16
RF12: Details of each property	4
Inquire about properties	10
Sprint deployment	4
Total estimated hours	**100**

- **Sprint 8 - Start date: 04/02/2025 End date: 11/02/2025 (88 hours)**

Task	Estimated hours
Meetings with the client	2
Tutoring meetings	1
Sprint documentation	4
Buffer for unforeseen events	10
Web application-level UX/UI adjustments 15	
Bug fixes	20
Final project documentation	36

2.4. Sprint 5 (Start date: 06/12/2024 end: 20/12/2024)

PLANNED TASKS:

- Training/Research
- RF7: Issuance of monthly rent settlement report
- RF9: Website homepage

- RF18: Simultaneous publication of properties on the website and RF10: Catalog of rental properties
- Sprint deployment
- Contingency buffer
- Sprint testing

Requirements prioritization (MoSCoW)

To manage the planning and replanning of tasks in this sprint, it has been applied the MoSCoW methodology to define priorities and ensure the achievement of the most critical objectives.

Must Have (Essential)

- RF7 (issuance of monthly rent settlement report): is critical for the financial management and transparency with the owners, ensuring a accurate tracking of payments and balances.

Should Have (Important but not critical)

- RF10 (rental property catalog): is fundamental to the experience user and visibility of properties, but in case of problems It could be adjusted in a later sprint.
- RF18 (simultaneous publication of properties on the website): is a functionality that optimizes ad loading, but its absence does not affect the basic system operation.

Could Have (Desirable if time permits)

- RF9 (website homepage): although it is an important feature The presentation of the system could be addressed if we have the available time.

Training/Research:

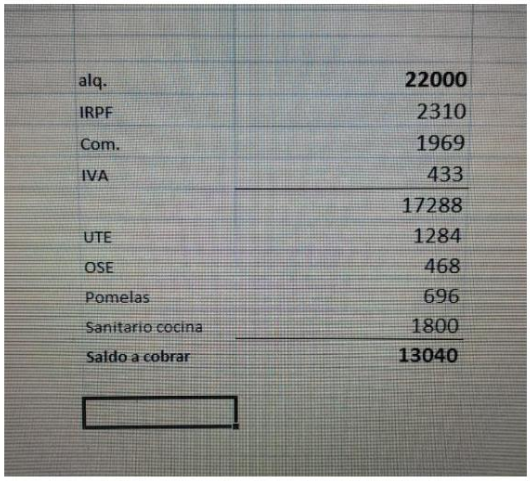
To begin working on the website in this sprint, we had to review React concepts.
Watching tutorials on YouTube and researching the Tailwind framework to get a better result in the visual section of the page.

RF7: Issuance of monthly rent settlement report:

To implement the functionality of issuing a monthly settlement report
Our rental client showed us the way in which they currently present these reports.
to its clients, which is based on Excel spreadsheets that indicate the various deductions that are applied to the initial rental amount and indicates the total balance paid to owner.

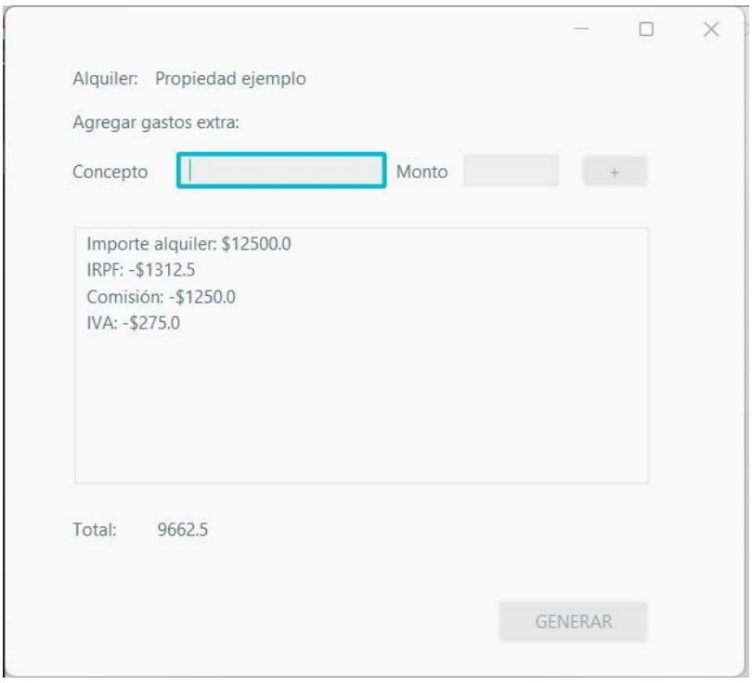
A rental agreement has an initial amount, which, if the rental agreement is not exempt from income tax, will be subject to a tax.
A 10.5% discount is applied to the initial amount (otherwise, no discount is applied). Furthermore,
The commission charged by the real estate agency for property management is deducted.
which corresponds to 10% of the rental amount. And finally, the following is also deducted:
VAT on the rental, which corresponds to 22% of the commission value.

After making all those deductions, the remaining value corresponds to the balance to be collected for the property owner.



alq.	**22000**
IRPF	2310
Com.	1969
IVA	433
	17288
UTE	1284
OSE	468
Pomelas	696
Sanitario cocina	1800
Saldo a cobrar	**13040**

We designed a window that displays listings of the initial rental amount, the
After various deductions are made to this amount, the total balance to be collected by the owner
and the option to add extra expenses to the list (when adding a new extra expense, the list will
It updates by adding it to the same and the total balance to be charged is also updated.
owner). Once the administrator finishes entering the expenses, press the button
Click “Generate” and a File Chooser will open where you specify the path where the report will be saved.
monthly settlement in PDF format.



RF9: Website homepage:
We have created a website using React and Tailwind CSS, with an attractive design
and fully responsive to adapt to both desktop computer screens
such as mobile devices.

Regarding the design, we have implemented a navbar within the header that serves as
page header. The header also includes a background image of the city in the
where the real estate agency is located, which adds a very representative visual touch and
It perfectly contextualizes the site. The background image adjusts dynamically according to the
screen size, thanks to the utilities of Tailwind CSS.

Following the header, we created a featured properties section. In this part of
On this page, we showcase some of the most relevant properties that the real estate agency has.
Available for rent. Each property is presented in a clear and organized manner.
using a clean design that leverages Tailwind CSS classes to maintain the
Visual consistency and ensuring a good browsing experience. This section is
designed to capture users' attention and allow them to explore the options in a way
efficient.

Further down the page, we added a contact section with a form that allows you to
Users can send a message directly to the real estate agency. This form is designed
to collect contact information, such as name, email address and message,
and is sent by email to the real estate team. This makes it easier
communication with potential customers is offered, and a quick and easy way to get in touch is provided.
in contact with the business.

RF18: Simultaneous publication of properties on the website and RF10: Catalog of rental properties:

We decided to combine these two requirements because they referred to the same thing, more than anything else.
to avoid ambiguity.

To fulfill these requirements, the following was done: from the catalog of
The website properties are linked to the management system's API and all the properties are retrieved.
Registered properties that are available for rent to be uploaded to the site.
In this way, when registering a new property in the internal system, the change will be reflected.
on the web simultaneously, and will always be kept up to date.

We successfully designed the website's property catalog according to what we had agreed upon.
For each property, our client is shown a photo, its description, and the price.
rent and the type of property.

Sprint deployment:

The website deployment on the Amazon Web platform has been successfully completed. Services (AWS). This process involved the configuration and deployment of resources necessary to ensure the site is fully operational and accessible through Internet.

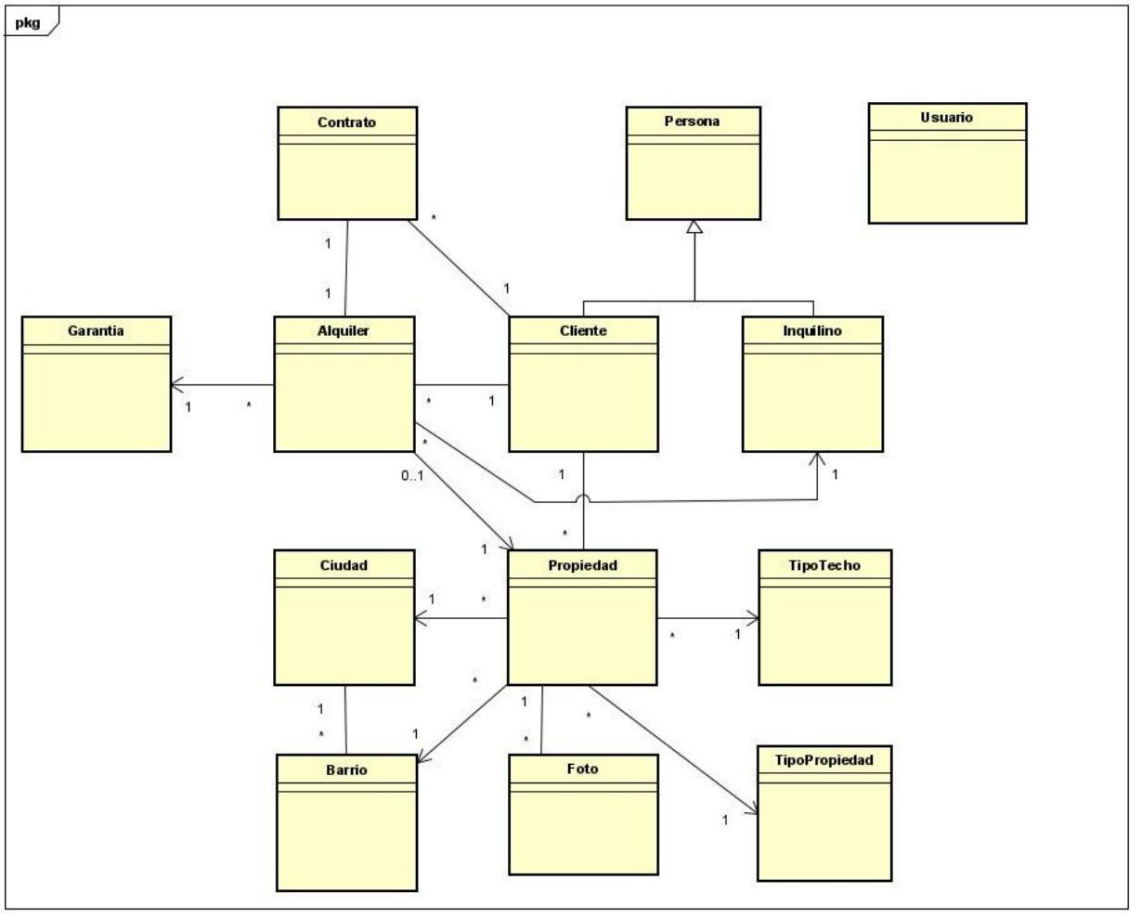
Contingency buffer:

We used a large part of the contingency buffer to continue improving the graphical interface of the internal management system.

LIST OF UNPLANNED TASKS

-Improvements to the visual presentation of the application.

PRELIMINARY DESIGN DOCUMENTS



SPRINT TESTING:

The development of the use cases that were tested in this sprint (CU009) can be found in the Annex 1 and the evidence of the testing is found in Annex 4.

MANAGEMENT TASKS/MEETINGS:

In the meeting with our client at the beginning of the sprint, we requested a report on monthly settlement to use as an example to implement RF7 and was consulted about what things I wanted to be present on the website's homepage.

He told us that he wanted a representative photo of Juan Lacaze and that
It will be explicitly stated that they have rental properties in that city.
We also asked him how he wanted the catalog to be displayed.
properties, to which he replied that his idea was to show a photo of each
property, its description, the rental amount and the type of property.

In the meeting with our tutor halfway through the sprint, we showed him the progress that
We had it both on the website and in the internal management system.

It should be clarified that in the effort analysis, despite having unified the
RF10 and RF18 requirements, we classify the hours we dedicate as follows
method: the hours dedicated to RF10 refer purely and exclusively to visual design
from the website catalog, while the hours dedicated to RF18 refer to the
the link that was made with it in the backend to hit the system's API
internal and load the properties.

STRESS ANALYSIS:

Task	**Estimated time**	**Real time**
Meetings with the client 4		2
Tutoring meetings	4	1.5
Sprint 4 Documentation		4

Training/Research 10		10
Sprint testing	4	4
Contingency buffer	10	8
RF7: Issuance of monthly rent settlement report	12	10.5
RF9: Website homepage	8	7
RF18: Publication in simultaneous listing of properties on the website	20	17.5
RF10: Catalog of rental properties	24	22
Sprint deployment	4	3
Total hours	104	89.5

The objective of this sprint was to implement the simultaneous publication of properties and start developing the real estate agency's website, including the homepage such as the property catalog. It was also intended to develop the

requirement for the issuance of monthly rent settlement reports and finally deploy the sprint.

RISK MONITORING:

No risks were presented in this sprint, neither those indicated in the preliminary project nor new ones.

CONCLUSIONS AND REPLANNING OF THE SPRINT STACK:

At the end of this sprint we can conclude that all the objectives were met.
They proposed it at the beginning of this within the stipulated time. Despite the fatigue
We closed the last one, which we had accumulated after programming non-stop all year.
2024 sprint with good feelings.

The next sprint will be structured as follows:

- **Sprint 6 – (Deliverable 5) - Start date: 21/12/2024 end: 19/01/2025 (99 hours)**

Tarea	Horas estimadas
Reuniones con el cliente	4
Reuniones de tutoría	4
Documentación del sprint	4
Capacitación/Investigación	10
Testing del sprint	4
Buffer de imprevistos	10
RF8: Stock de propiedades de cliente	8
RF4: Notificaciones de vencimientos	20
RF5: Notificaciones de atrasos en los pagos y cambio de nombre de servicios	16
RF14: Pestaña de servicios	8
RF15: Pestaña de contacto	6
RF16: Pie de página	1
Deploy del sprint	4
Total horas estimadas	99

2.5. Sprint 6 (Start date: 21/12/2024 end: 19/01/2025)

PLANNED TASKS:

• Training/Research • Sprint Testing

• Contingency buffer • RF8:

Stock of client properties

• RF4: Expiration notifications

• RF5: Notifications of late payments and changes of service names
• RF14: Services tab
• RF15: Contact tab
• RF16: Footer
• Sprint deployment

Requirements prioritization (MoSCoW)

To manage the planning and replanning of tasks in this sprint, it has been applied the MoSCoW methodology to define priorities and ensure the achievement of the most critical objectives.

Must Have (Essential)

• RF4 (expiration notifications): is essential to ensure management efficient contract management and avoid delays in renewal or termination rentals.
• RF5 (notifications of late payments and change of service name): critical for financial management and avoiding problems with owners and tenants.
• RF8 (customer property stock): essential for managing correctly the availability of properties and avoid inconsistencies in the unit allocation.

Should Have (Important but not critical)

• RF14 (services tab): provides relevant information to users, but It does not directly affect the main operation.
• RF15 (contact tab): facilitates communication with customers.

Could Have (Desirable if time permits)

• RF16 (footer): improves website navigation and provides links of interest, but its implementation can be postponed without affecting the main functionality.

Training/Research:

In order to implement the notification requirements, we had to train ourselves on how to make it so that automatic emails are sent when the system detects the expiration of a contract, a delay in rent payment, or a change of name from the utility bills for a property after a certain period of time. Then After investigating alternatives, we decided on the JavaMailSender interface, which is part of from the Spring Framework, which facilitates sending emails using the JavaMail API library.

As our client requested that the emails be sent on specific days and times We had to investigate how to schedule the tasks responsible for that functionality to ensure they ran at the right time. For this, we decided by using the @Scheduled annotation from the Spring Framework package, which is used to schedule the automatic execution of tasks at specific intervals.

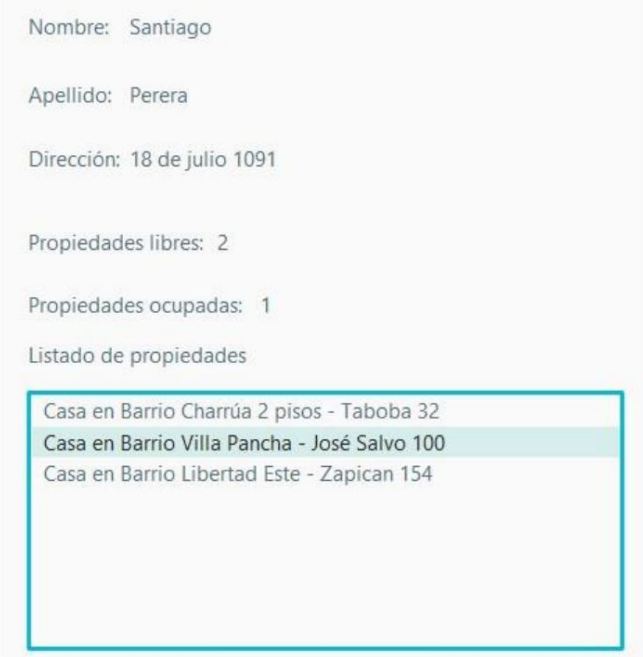
Contingency buffer:

We used up our entire contingency buffer (and a few hours more than we were going to dedicate to the notification requirements) developing the logic of the RF3.5: Rental tracking.

RF8: Stock of client properties:

To fulfill each client's property stock requirement, we opted for

Include labels in the customer record, whose values are loaded based on subtracting the quantity from rentals to the total number of client properties (to obtain the amount of free properties) and based on obtaining the amount of rentals (to obtain the amount (of occupied properties). These are also automatically updated when Properties are added or removed using a callback.



RF4: Expiration notifications:

This requirement was only partially implemented because we had to dedicate several hours to the unplanned task of tracking rentals, as we needed to have much of that logic implemented in order to later develop the notification functionalities.

RF5: Notifications of late payments and changes of service names:

We implemented this requirement partially, but we managed to complete it in its
in full and leave the late payment notifications working, but we left
partially developed the logic for name change notifications
services.

Our client explicitly requested that they would like to receive late payment notifications.
Payments are sent via email from the real estate agency every Monday at 10 AM.
which would specify the name of the tenant who is behind on payments, address
of the property you are renting, your phone number and the number of months of
the delay. He told us that this way the secretary would verify this email, call
to the tenants by phone to regularize their situation and, once that has been done
Once the call was made, the secretary would mark the email as read.

Based on our client's request, we decided to use JavaMailSender to send the
emails from the system to the real estate agency's email address,
configuring a method that iterates through the rentals looking for any with 1 or more
months of payment delays and sending the corresponding email for each one. Said
The method has the @Scheduled annotation which makes the function run every Monday
at 10 AM sharp.



LIST OF UNPLANNED TASKS

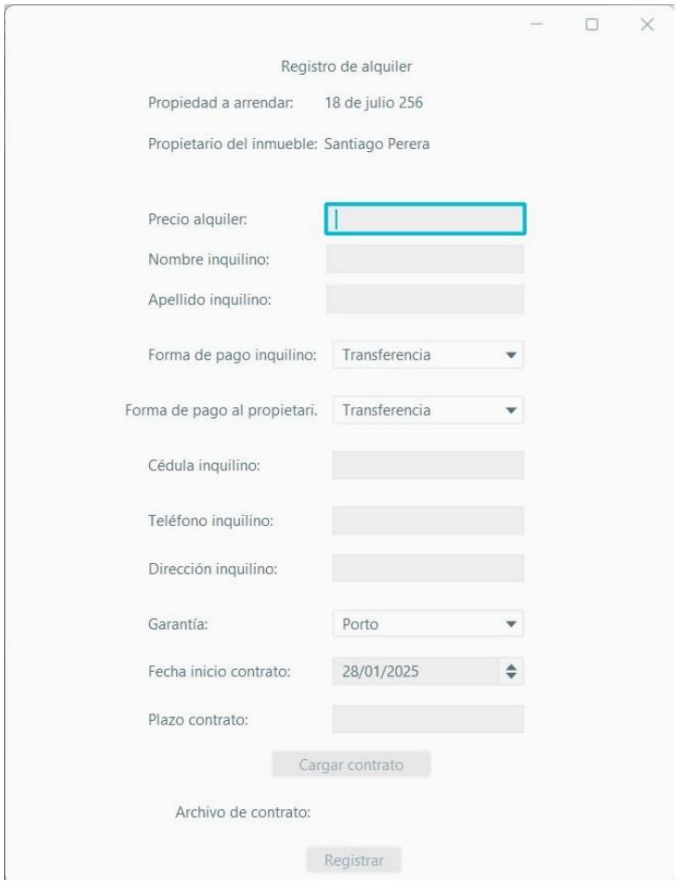
-RF3.5: Rental tracking (partial)

RF3.5: Rental tracking:

To meet the rental tracking requirement, the first thing we did was create
the rental registration window, in which our client asked us to request the
the amount of rent, the tenant's details, and how the tenant will pay you

the real estate agency (cash at the counter or by bank transfer), the way in which
The real estate agency will pay the homeowner what they are entitled to for each month of rent (can also be in cash or by bank transfer), the type of guarantee already
that the contract, the rental period in years, and the form also be included
date on which the contract was signed. We had already partially implemented the
RF3.1, which comprised a form for registering contracts, therefore the only
What we had to do was move that logic to the rental registration window and perform minor modifications.

It should be clarified that the completion of the contract registration requirement was planned for the next sprint, but since we had to unify it with rental tracking
We decided to finish it now in this sprint.



RF14: Services tab:

In this requirement, a new section was developed on the main page that highlights the three main services offered by the real estate agency, in addition to the option of renting a property. This section aims to inform the user about the full range of services available, facilitating your exploration and decision.



RF15: Contact tab:

We created a section on the homepage so users can communicate and Inquire about available properties or services offered. This section includes a a form that can be sent via Gmail or WhatsApp, providing options for quick and accessible contact.



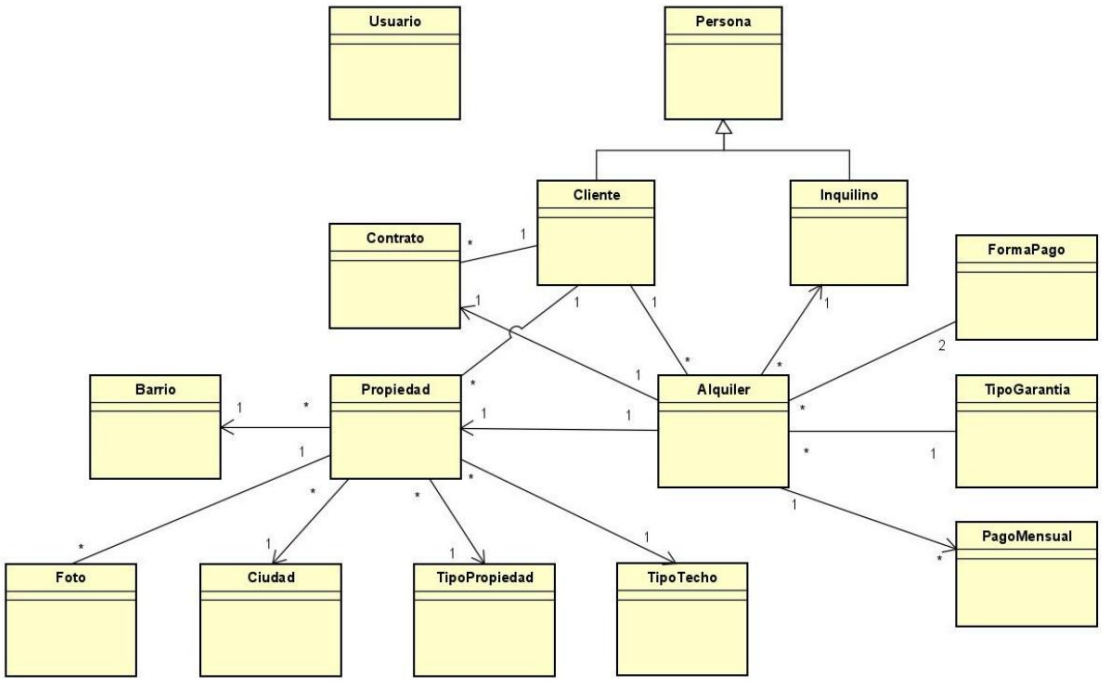
RF16: Footer:

A footer was developed that displays the real estate agency's address, and a button that opens the WhatsApp chat with the real estate agency and a button that redirects to its Facebook page.

Sprint deployment:

We successfully deployed the internal system's backend on AWS and decided postpone the website rollout to the next sprint because we prefer to dedicate more time to it. time to develop RF3.5, as we considered it a vital part of the project in the which meant we couldn't risk having mistakes in the days leading up to the final delivery.

PRELIMINARY DESIGN DOCUMENTS:



SPRINT TESTING

The development of the use cases that were tested in this sprint (CU010 and CU011)

They are found in Annex 1 and the evidence of the testing is found in Annex 4.

MANAGEMENT TASKS/MEETINGS

In the sprint kickoff meeting with our client, we asked them how they

would like to receive notifications of late payments from tenants, to which we

He commented that he would find it practical if the notifications were sent directly to his email.

the real estate agency (which the secretary reviews periodically). She also clarified that she

I would like late payment notifications to be sent every Monday.

at 10 AM (the time when the secretary starts her workday), so that it begins

the week, having a clear idea of who to call to get them up to date with their payments

rentals. We asked him about the content of the notification email and he told us that he

I would like an email to be sent for each tenant in arrears, including their details.

personal details (full name and phone number), the address of the property that is

renting and the number of months of arrears in payments. In this way, once

that the secretary notifies the client by phone that they are late in their payment, would mark the email marked as read.

Our client was asked to explain again how the system currently works.
rental payment process at the real estate agency to be able to implement it in the best way
The requirement for monitoring rentals may be necessary.

The notary reminded us that the process begins with the registration of the contract of
lease, which sets out the terms of the rental, including the amount
monthly, the tenant's details, the payment method, and how it will be paid to the
owner. During this registration, the type of contract guarantee is also defined, the
rental duration in years and the date the agreement was signed.

Each month, the tenant must pay the rent to the real estate agency. This payment can be made
In two ways: cash at the counter or by bank transfer. In the case of
For a cash payment, the tenant goes to the real estate agency, hands over the money, and the
The secretary hands you a hand-filled rent receipt. If payment is made by
The transfer, the real estate agency must verify the credit in its bank account before
prepare the rent receipt.

After receiving payment from the tenant, the real estate agency must settle the corresponding amount.
owner. This payment can also be made in cash or by bank transfer.
If payment is made in cash, the real estate agency withdraws the money from its bank account.
The bank transfer is made and the customer receives the payment in cash at the office. Upon receiving payment, the owner
He signs a receipt acknowledging receipt of payment. If the settlement is made by
The transfer, the real estate agency sends the money and, once the transfer is confirmed, the money is sent.
Upon accreditation, the owner is notified and given a payment receipt.

Furthermore, regarding the requirement for rental monitoring, our client warned us about ultimately not wanting the option to register payments of services because this would create an unnecessary workload for your secretary.

We also asked him how he wanted to access each client's property stock,
To which he replied that he would like to have this information visible when opening a record of a client, which would include the number of vacant and occupied properties of the same.

Regarding the service tab requirement, the client detailed the types of service offered by the real estate agency, and we agreed to show you a sketch of what they would look like submitted on the web for him to approve before implementation.

In the meeting with our tutor, we focused on showing him the progress in RF5 and discussing Why did we decide to bring forward RF3.5 for this sprint and partially implement the notification requirements.

STRESS ANALYSIS

Task	**Estimated time**	**Real time**
Meetings with the client 4		3.5
Tutoring meetings	4	2
Sprint 4 Documentation		4

Training/Research 10		8
Sprint testing	4	3.5
Contingency buffer	10	30
RF8: Customer property stock	8	4
RF4: Expiration notifications	20	10
RF5: Notifications of payment delays and name change of services	16	13.5
RF14: Services Tab 8		7
RF15: Contact tab 6		6
RF16: Footer	1	1
Sprint deployment	4	4

Total hours	**99**	**96.5**

The objective of this sprint was to implement the notification requirements
(RF4 and RF5), which were partially implemented because we decided
to bring forward RF3.5 for this sprint (initially that requirement was going to be implemented in the next sprint) due to its importance to the project, while also
We needed to implement its logic in order to then develop the functionalities of the Notifications. The hours we dedicated to RF3.5 correspond to the hours we had left over.
of RF4 and RF5 (because we only partially implemented them), which
we place it in our contingency buffer.

This sprint also aimed to implement the property stock.
for each client and also on the website the goal was to implement the tab of
services, the contact tab and the footer.

RISK MONITORING

In this sprint, a new risk arose, which was that after the sprint had started,
The development team noticed that a task from a later sprint would need to be moved forward.
This (RF3.5). What was done was to implement the notification requirements in a way
partial and postpone the completion of these to the next sprint in order to be able to partial
dedicate more time to RF3.5.

CONCLUSIONS AND REPLANNING OF THE SPRINT STACK:

At the end of this sprint we can conclude that without considering the requirements of the Notifications: all requirements were fully met (and it was even possible to to bring forward RF3.4 and despite the unexpected adjustments that had to be made, the The project timeline was not significantly affected.

The next sprint will be structured as follows:

- **Sprint 7 – (Deliverable 6) - Start date: 20/01/2025 end: 03/02/2025 (100 hours)**

Task	**Estimated hours**
Meetings with the client	4
Tutoring meetings	4
Sprint documentation	4
Training/Research	10
Buffer for unforeseen events	10
RF11: Property Finder	18
RF4: Notifications of 4 deadlines (end)	
RF5: Delay notifications in the payments and name change of services(end)	4

Final testing	16
RF12: Details of each property	4
RF13: Property Inquiry	10
Sprint deployment	4
Total estimated hours	**92**

2.6. Sprint 7 (Start date: 20/01/2025 end: 03/02/2025)

PLANNED TASKS:

- Training/Research
- Sprint testing
- Contingency buffer •

RF11: Property finder

- RF4: Expiration notifications (finalize) • RF5:

Notifications of late payments and change of service name

(finish)

- RF12: Details of each property
- RF13: Property Query • Final Testing

• Sprint deployment

Requirements prioritization (MoSCoW)

To manage the planning and replanning of tasks in this sprint, it has been applied the MoSCoW methodology to define priorities and ensure the achievement of the most critical objectives.

Must Have (Essential)

• RF4 (expiration notifications (finalize)): is essential for the management of contracts and avoid problems with customers.

• RF5 (notifications of late payments and change of service name) (finalize): key to financial control and efficient management of properties.

• RF11 (property search engine): allows users to find properties easily, improving the customer experience and the usability of the system.

Should Have (Important but not critical)

• RF12 (details of each property): adds value to the property catalog, but its absence does not impede the main operation of the system.

Could Have (Desirable if time permits)

• RF13 (property query): facilitates interaction with clients, but it It could be postponed to the next sprint without affecting key functionalities.

Training/Research:

For this sprint we had to train ourselves on AWS about how we could store the Photos of the properties in a folder on the server (using a Bucket) and access

These are available from the web.

Contingency buffer:

A large part of the hours allocated to the contingency buffer were used to solve how to make the property photos appear on the website and to carry out the website deployment, as it took us longer than estimated in a principle.

RF11: Property search engine:

For this requirement, we have implemented the option to filter the entire list of Available properties. Filters include neighborhood, property type, number of rooms, number of parking spaces, and whether pets are allowed. Additionally, when applying multiple filters, only properties that simultaneously meet all the criteria will be displayed Selected criteria.



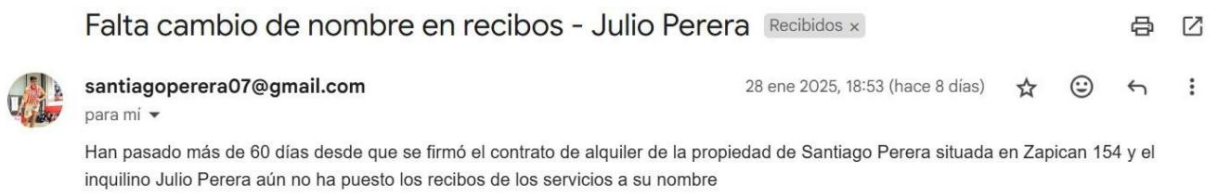
RF4: Expiration notifications (end):

Our client asked us to let them send a notification when a contract...
The lease expires, which would specify the name of the property owner,
property address and tenant's name. Do you want these notifications sent to you?
They arrive at the real estate agency's email every day at 9 AM.

Based on our client's request, we decided to use JavaMailSender to send the
emails from the system to the real estate agency's email address,
setting up a method that goes through the rental contracts looking for any that
expires on that same day. This method has the @Scheduled annotation which makes the
The function runs every day at 9 AM sharp.

RF5: Notifications of late payments and changes of service names (finish):

We successfully completed RF5, which we had started in the previous sprint.
The client had requested that they would like to receive a notification if, after 60 days of
Having started a rental agreement, the tenant had not yet changed the OSE and UTE utility bills to
his name. What we did was implement a method in the rental service, the
which goes through the list of rentals and checks if there is any rental agreement that has
initiated more than 60 days ago that does not yet have the account holder for the UTE and OSE bills
to the tenant. This method has the @Scheduled annotation which makes the function
Run every Monday at 9 AM sharp and within its logic it uses
JavaMailSender to send notification emails.



RF12: Details of each property:

This requirement implemented an improvement to the available properties page. to rent, adding a button that, when selected, displays the complete information of the property with all its details.

RF13: Property Inquiry:

Several hours were dedicated to implementing this requirement, but ultimately We agreed with our client that it would not be necessary because through the form Customers can also contact the company to inquire about specific properties. messages.

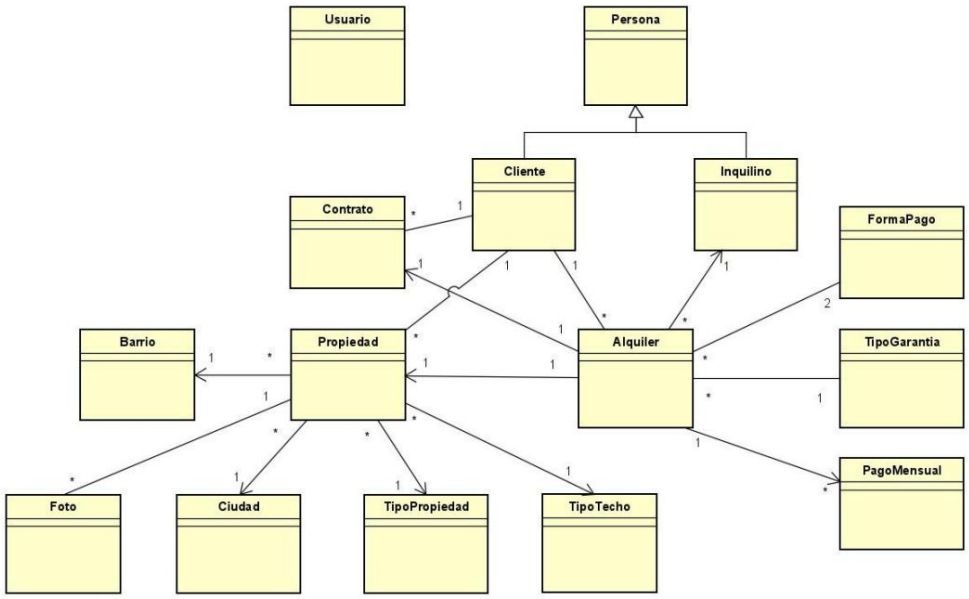
In summary, this requirement was unified with RF15: Contact tab.



Sprint deployment:

In this sprint we focused on implementing the website deployment, which is explained in the Annex “Deployment Manual”.

PRELIMINARY DESIGN DOCUMENTS:



SPRINT TESTING

The development of the use cases that were tested in this sprint (CU012, CU013 and CU014) They are found in Annex 1 and the evidence of the testing is found in Annex 4.

MANAGEMENT TASKS/MEETINGS

In the meeting with our tutor, we focused mainly on demonstrating the functionality.

Once the requirements were fully completed, we began planning how we were going to

to organize ourselves to prepare the final document.

STRESS ANALYSIS

Task	**Estimated time**	**Real time**
Meetings with the client	4	2
Tutoring meetings	4	3
Sprint documentation	4	4
Training/Research	10	8.5
Contingency buffer	10	6
RF11: Property Finder	18	15.5
RF4: Expiration notifications (finalize)	4	4

RF5: Notifications of payment delays and change service name (finish)	4	4
RF12: Details of each property	4	3.5
Sprint deployment	4	4
RF13: Property Inquiry	10	9
Final testing	16	16
Total hours	92	79.5

This sprint was planned for 92 hours, of which 79.5 were executed. The following were completed: Key features such as the property search engine, expiration notifications, and Delays, property details, and property inquiries. Meetings and training They required less time than expected, optimizing execution. Testing and deployment were They performed as planned, and the contingency buffer was not exhausted. Overall, the sprint It was efficient, meeting the objectives without affecting quality.

RISK MONITORING

In this sprint, no risks were identified, neither those defined in the preliminary project nor new.

CONCLUSIONS AND REPLANNING OF SPRINTS

At the end of this sprint we can conclude that all the objectives were met.
They proposed at the beginning of this within the stipulated time.

The next sprint consisted of the following tasks:

- **Sprint 8 - Fecha inicio: 04/02/2025 fin: 11/02/2025 (88 horas)**

Tarea	**Horas estimadas**
Reuniones con el cliente	2
Reuniones de tutoría	1
Documentación del sprint	4
Buffer para imprevistos	10
Ajustes UX/UI a nivel de aplicación web	15
Corrección de bugs	20
Documentación final de proyecto	36

2.7. Sprint 8 (Start date: 04/02/2025 end: 11/02/2025)

PLANNED TASKS:

- UX/UI adjustments at the web application level
- Bug fixes
- Buffer for unforeseen events

UX/UI adjustments at the web application level:

Several adjustments were made to the user experience (UX) and user interface (UI). of the web application with the aim of improving navigation and usability. First In this place, the layout of the property catalog was optimized, centering the elements to to achieve a more balanced and aesthetically pleasing presentation.

Adjustments were also made to the size and spacing of the property cards, which It allowed for better readability. Likewise, the color contrast and typography were improved. ensuring greater visual comfort for users.

Bug fixes:

We were having trouble uploading property images to the website because... When uploading images to the API using only the desktop application We stored the file path on the local PC, therefore, when trying to paste it from the website The API to load that route couldn't find anything. We tried to fix it, but because it was so...

Closer to the delivery date, we decided to leave it as it was to avoid errors elsewhere.
of the project.

Buffer for unforeseen events

We do not need to use the time allocated to the contingency buffer in this sprint.

MANAGEMENT TASKS/MEETINGS

In meetings with our tutor, we spend most of the time asking questions.
Regarding the progress of the documentation and trying to fix the error in uploading the photos
to the website. In the meeting with our client, we showed them the almost finished product and
He was very satisfied with the result.

STRESS ANALYSIS

Task	**Estimated time**	**Real time**
Meetings with the client	2	2

Tutoring meetings	1	4
Sprint documentation	4	2
Buffer for unforeseen events	10	0
UX/UI adjustments at the web application level	15	8
Bug fixes	20	18.5
Project end documentation	36	33

The objective of this sprint was to finalize the project documentation and fix bugs.

and polish visual details in the web application. We can affirm that the objectives are

They fully complied.

RISK MONITORING

The following risks were identified during this sprint:

- **R6: Poor estimation of the time required to perform each task.** This risk affects us.

It was presented within the estimated time for tutoring meetings, we estimated 1 hour, but

We finally ended up dedicating 4 hours because we needed help to get it done.

They will upload the photos to the website.

3. Project Summary

3.1. Project evolution

3.1.1. Final scope of the project

Below is a list of all the features that were implemented.
in the project, classified among functionalities of the real estate company's internal system and website functionalities.

Internal system functionalities:

- **Login:** The system has an authentication screen that restricts access exclusively for registered administrators. A form has been implemented for this purpose. where administrators enter their previously registered username and password.

Authentication validates the entered credentials by comparing them with those stored in the database. If the data is correct, access to the system is granted with the permissions corresponding; otherwise, an error message is displayed indicating that the
The credentials are incorrect.

- **List of owner clients:** Upon logging into the system, the list of Registered owner clients, having the option to select one and press the “View” button "file" to display its detailed file or press the "Add client" button to register a new owner client in the system.

- **Registration of owner clients:** The system allows the registration of owner clients.

During registration, relevant owner data is stored, including first and last name,

address, telephone number and ID number.

In addition, a space in the database is automatically generated and assigned to each owner.

where information about your properties, contracts and associated rentals will be organized.

- **View a specific owner's profile:** When you select a client from the list of

customers and press the “View profile” button, the owner's profile is displayed which includes:

-Customer's personal data (Name, Surname, Address, Telephone number and ID number).

-List of registered properties.

-Historical list of registered rental contracts.

- **Add property to a customer owner:** On a customer's record, the system

It allows you to register new rental properties associated with this one. To complete the registration, the

The user must enter the key property details.

Information required for registration:

-Description

-Address

-Neighborhood

-City

-Type of property (House, Apartment, Commercial premises or Garage)

-Photo/photos of the property

-Number of rooms

-Number of bedrooms

-Number of plants

-Number of bathrooms

-Total square meters

-Square meters built

-Square meters of land

-UTE Owner

-OSE Holder

-Type of roof (Sheet metal or Flat bar)

-Number of garages

-Does it accept pets or not?

- **To delete a customer's property:** In the customer's record, when selecting a Within the property listing there is a button that allows the deletion of said property (previously displaying a confirmation window before deleting it), in case the The owner no longer wishes to rent it out. This functionality ensures that the inventory of properties available in the real estate agency are kept up-to-date and accurate.

Considerations:

-A property cannot be removed if it has an active rental agreement.

-Once deregistered, the property is no longer available in the system for future listings. rental operations.

- **Rental and contract registration:** In the property listing on a client's profile, There is the option to select a property and register a rental of it, which is done through a form in which the following data is requested:

-Base rental price

-Tenant's name

-Tenant's last name

-Method of payment by the tenant (cash or bank transfer)

-How the real estate agency pays the property owner (cash or bank transfer)
banking)

-Tenant's ID

-Tenant's phone number

-Tenant's address

-Rental guarantee

-Contract start date

-Contract term

-Contract file in PDF format

Once the “Register” button is pressed, the system validates that the data is correct. If
If the data is correct, the rental is successfully registered in the system; otherwise,
A sign is displayed indicating which field contained invalid data.

- **Rental listings:** On an owner's profile, you can press the "View" button.
"rentals" to display the list of rentals. When you select a rental from the list,
There is the option to open the rental tracking window, choose a month, and generate
a payment receipt or generate a monthly settlement receipt.

- **Generate rental payment receipts:** In the rental listing window, the system
It allows you to select a rental from the list, choose a month in a combo box, and then press the button
“Generate payment receipt” opens the file explorer and the administrator chooses the path where
You wish to save the receipt in PDF format, which indicates the tenant's name and the rental month.
who pays, the amount paid, and the date the receipt was issued. At the bottom, leave a
space requesting our client's signature to validate the receipt.

- **Generate monthly settlement reports for owner clients:** In the

In the rental listing window, the system allows you to select a rental from the list, choose a month of the combobox and when you press the “Generate monthly settlement” button a window in which the address of the rental property is displayed at the top and Below is a list of deductions made from the initial rental price.
This window allows you to add extra expenses, detailing the concept and amount of each. Finally, below the list of expenses, the total price to be paid to the property owner is shown. which is updated every time a new expense is added.

Once the administrator has finished adding all the extra expenses, press the button “GENERATE” and the file explorer opens, in which you will select the path where the will store the PDF document of the monthly settlement for the owner client.

This document indicates the amount to be paid to the client/owner, the name of the tenant, the property address, the month to which the settlement corresponds, the date of Issuance of the receipt, the list of basic and extra expenses, and below a space for the signature owner client.

- **Customer property stock:** When you open a customer's record, the system displays in

The screen displays how many properties are occupied and how many are available. When adding or deleting properties and when assigning or removing rents to a property, the values displayed on the screen They are updated automatically.

- **Expiration notifications:** Every day at 9 AM, the system checks if there is an

For each rental agreement that expires on the current date, send an email to the real estate agency's email address detailing the address of the rented property, the name of the property owner, the name of the tenant, and the date of expiration of the rental agreement (current date).

- **Notifications of late payments and changes of service names:** All

On Mondays at 10 AM, the system checks if there are any tenants behind on their rent payments and therefore Each of them sends an email to the real estate agency's email address, in the

which specifies the name of the tenant who is behind on payments, address of the property you are renting, your phone number and the number of months you are in arrears.

Additionally, every day at 9 AM, the system scans the rental listings looking for rentals. that started 60 days ago or more and the UTE and OSE receipts for the property are not yet available in the tenant's name. In the event of rentals that meet these conditions, the system send an email for each of them to the real estate agency's email address notifying the situation, indicating the full name of the property owner, address of the rental property and full name of the tenant.

Website functionalities:

- **Site presentation:** The website's homepage has a section with a photo from the real estate agency and a presentation text about it.

- **Rental property catalog:** The website has a tab where you can find They present the rental properties in the form of a catalog with photos and a brief description below each one.

- **Property search engine:** The website has a property search engine that allows filter by different criteria (neighborhood, type of property, number of rooms, number of garages and whether or not pets are allowed).

- **Details of each property:** In the property catalog, each property has a “See more” button, which when pressed displays all the property specifications and It gives the option to close it to return to the initial view.

- **Property Inquiry/Contact Tab:** At the bottom of the page At the top of the website there is a form that allows users to contact the real estate agency via email or WhatsApp, allowing you to enter a message, your full name and your email address (in case (If I choose to send by email, this information is ignored if I choose to send by WhatsApp).

- **Services tab:** The website's main page has a section where you can

They detail the services offered by the real estate agency.

- **Footer:** At the bottom of the website there is a footer that contains hyperlinks

to the real estate agency's social media accounts and its address.

3.1.2. Deviations from the preliminary design

In the preliminary draft, the development team agreed that in case several tasks are delayed more than planned to implement, with the goal of delivering the best possible product to Given the delivery date, priority would be given to implementing the requirements listed as "MANDATORY" items would be used, and "OPTIONAL" or "DESIRABLE" items would be excluded.

For reasons explained earlier in this document, the development team decided against implementing these functional requirements in the system:

- **RF17: Online chat.**
- **RF18: Simultaneous property listing. • RNF7: Instagram account creation.**
- **RNF8: Hiring Instagram advertising service.**
- **RNF9: Creation of advertising reel.**

Clarification: RF18 was partially implemented; properties are only published.

When users register on the website, their information is not published on the social media platforms of the company. real estate.

3.1.3. Difficulties during development

During the development of the project we encountered the following difficulties:

- **Poor time estimation:** As reflected in the analysis tables of
  Due to the effort required in each sprint, some requirements took longer than expected.
  due to its complexity or unforeseen technical issues.
- **Lack of team experience:** Both developers had little experience in
  projects of this type, which slowed development on several occasions.
- **Property photo storage:** There were difficulties in management and
  image storage in the database.
- **Lack of clarity in the details of the functionalities in advance:** In some
  In some cases, not all aspects of the functionalities were defined before starting.
  its development, which generated delays and modifications on the fly.
- **Problems with backend and frontend integration:** During the connection between
  In both the backend and the frontend, errors were found in the communication with the API,
  which required additional adjustments.
- **Lack of advance testing:** In some cases, errors were detected late in
  the development process, which led to rework and adjustments that could have
  avoided with a more rigorous testing strategy from the start.

3.2. Project Management

3.2.1. Working Methodology

For the development of the real estate company's internal management system and website, the following was used:
an adaptation of Scrum, an agile framework that enables incremental value delivery and facilitates
managing complexity.

The project was organized into 2-week sprints, where increments were developed.
functional aspects of the system. During each sprint, the team held daily meetings (Daily
Scrum) to report progress, plan the day, and resolve impediments, ensuring a
fluid communication and alignment in tasks.

There was only one sprint, which lasted 4 weeks, because the developers weren't going to be available.
available within that timeframe, so a longer sprint was planned to reach
to complete all planned tasks.

At the end of each sprint, a sprint review was conducted, in which the work was presented.
developed for stakeholders to receive feedback and adjust the plan if necessary.
In addition, a sprint retrospective was conducted, where the team identified opportunities to
It improved and adjusted its processes to optimize performance in future iterations.

The entire development was carried out according to the preliminary design, respecting both
The organization into sprints, such as the distribution of tasks among developers. This
It allowed flexibility in the face of change, continuous improvement, and incremental deliveries, ensuring a
efficient development aligned with customer requirements.

3.2.2. Management tools used

For the development of this project, various planning tools were used,
Monitoring team progress and collaboration:

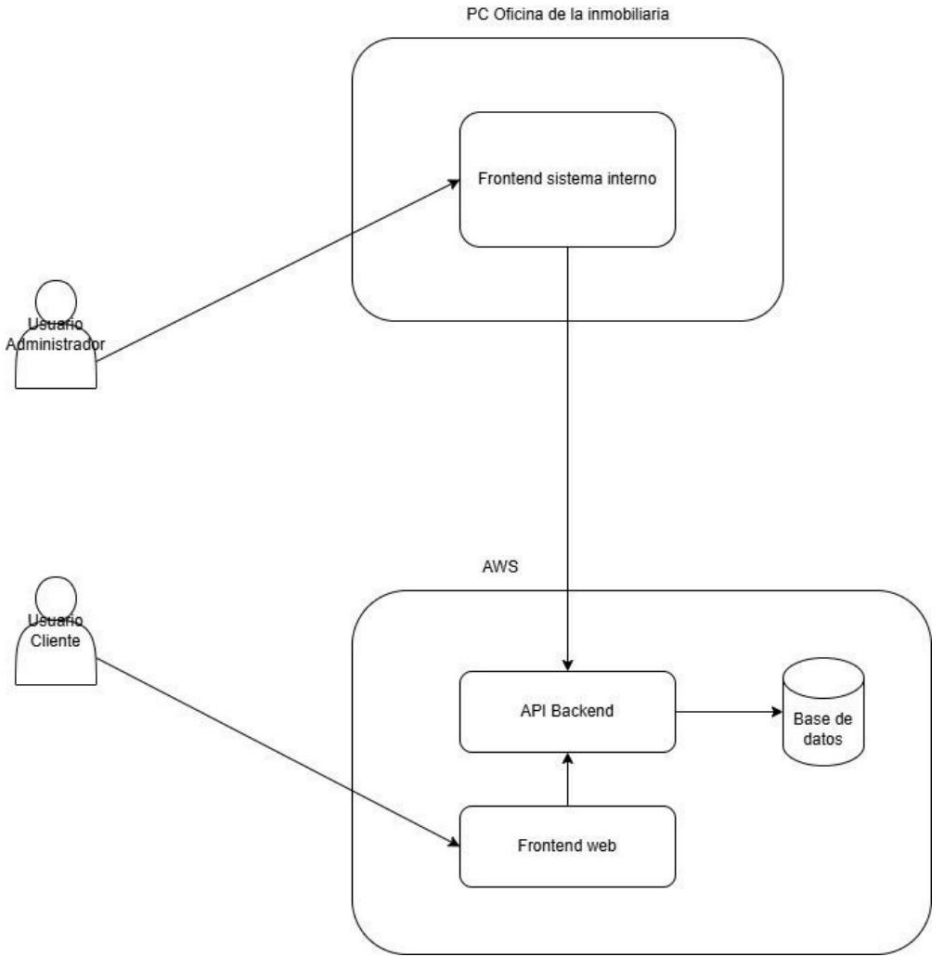
WhatsApp: Quick communication and resolution of doubts between team members,
in addition to constant communication with the client.

Discord: Video calls and daily coordination among developers.

Microsoft Teams: Meetings with the project tutor and document storage.

3.3. Solution design

3.3.1. Final Architecture



For our real estate project's API, we decided to use a microservices architecture to ensure each function is well-organized and easier to maintain and scale. We divided the system into several microservices, each focused on a specific aspect, allowing us to develop and improve each component without affecting the others.

Each one has its own business logic and its respective DAO, which helps us optimize database queries and maintain cleaner, more structured code.

We decided to implement separate microservices to handle the different key entities of the system. We have a customer microservice, responsible for managing all the information of the owners and their personal data. We also have a tenant microservice, which stores and manages the information of those who rent the properties.

The properties microservice centralizes the management of properties, while the property types microservice is responsible for defining and classifying the different categories of available properties.
In addition, the neighborhood and city microservices allow properties to be geographically organized within the system.

For managing rentals, we have a dedicated microservice that handles lease agreements, including the relationship between landlords and tenants. We have also developed a contracts microservice that stores the terms of each agreement in the database.

The payment system is decentralized into several microservices: the monthly payments microservice, which records and manages periodic rent payments, and the payment methods microservice, which defines the methods available for making transactions.

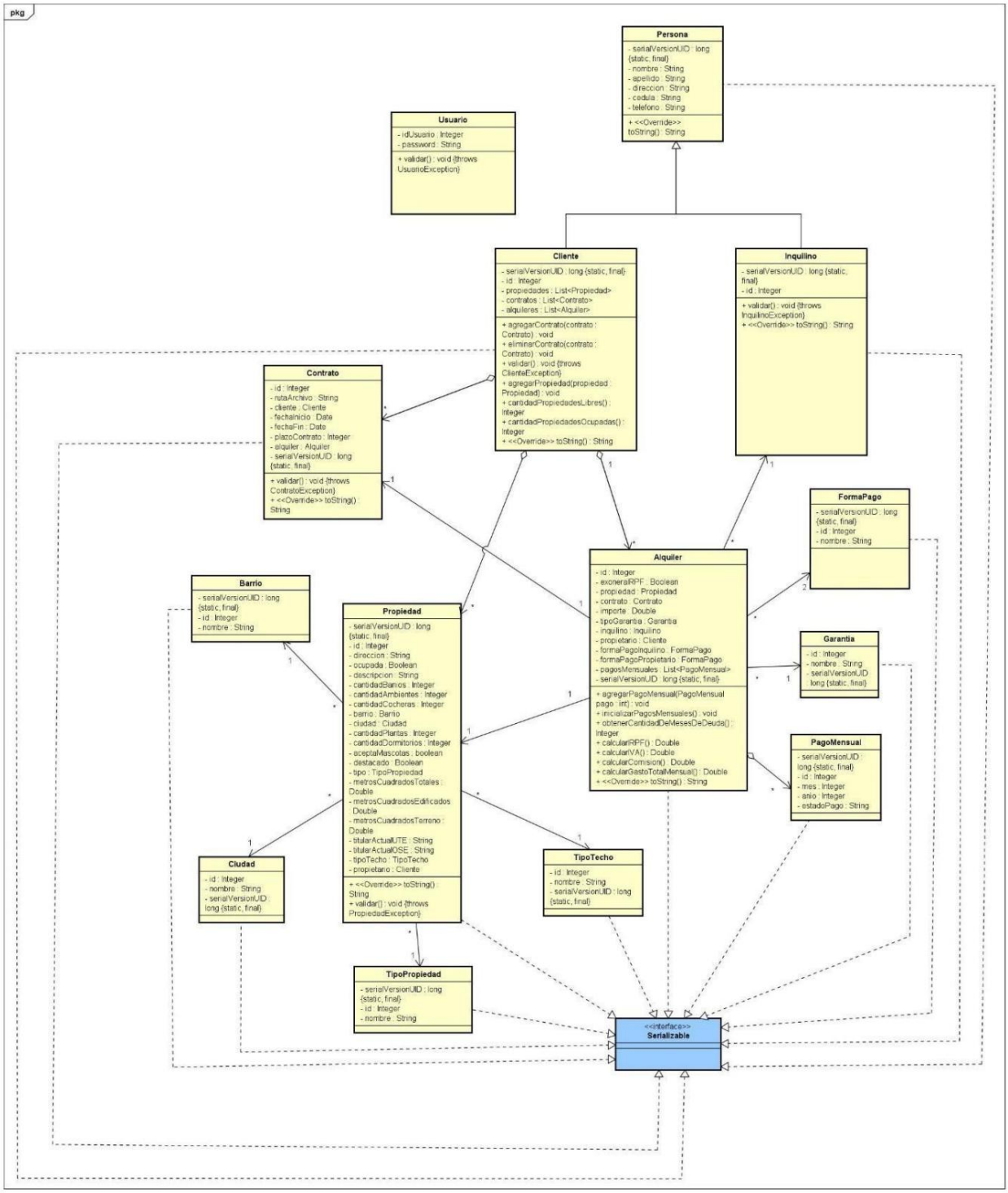
In addition, we incorporated a roof type microservice, which classifies properties according to their structural characteristics.

Thanks to this modular architecture, the system is more flexible and scalable, allowing us to continue adding improvements and new features in the future without complications.

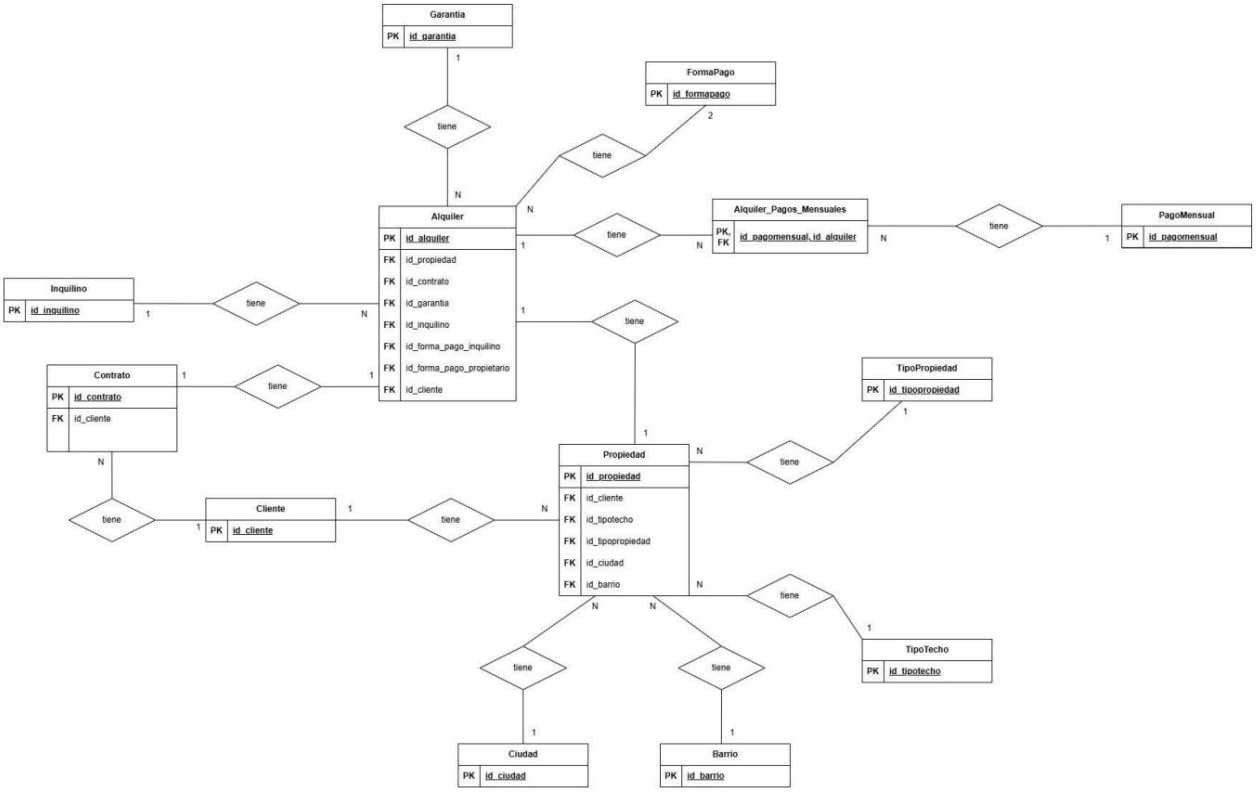
3.3.2. Use case diagram



3.3.3. Class Diagram



3.3.4. MER Final



Clarification: The attributes were not included in their entirety so that the diagram

To make it more readable, only the primary and foreign keys were included.

3.3.5. Technologies used

Technologies used for the system internal (frontend of desk - Administrators):

- **Java:** The system's main programming language.
- **Java Swing:** Creation of the graphical interface.

Technologies used for the web application (web frontend – Clients):

- **HTML:** Structure of web pages.
- **CSS:** Stylization and responsive design.
- **JavaScript:** Website interactivity.
- **React.js:** Library for building the user interface.
- **Tailwind CSS:** A CSS framework for fast and efficient styling.

Technologies used for the backend (which is consumed by the internal application and by the web application):

- **Spring Boot:** Framework used for backend development. • **Spring Data JPA:** Database persistence management.
- **MySQL:** Relational database used to store information. • **Hibernate:** JPA implementation for object-relational mapping (ORM).

3.4. Evidence of plan compliance

3.4.1. SQA/Testing Plan

Throughout the development of the project, the guidelines established in the plan were met. of quality, ensuring a functional, tested and well-documented product.

Documentation: The format provided in University document 302 was followed. ensuring clear and structured documentation that supports the development of the system.

Testing: As evidenced in the sprint documentation and use cases, At the end of each sprint, tests were performed to verify the correct functioning of the implemented functionalities. Black box tests were applied to validate the 126

compliance with functional requirements and white box testing to optimize the performance and detect logical errors from early stages, thus ensuring the quality of software.

Development: Standards and best practices were implemented to ensure code Clean, maintainable, and scalable. CamelCase was used for variable naming and methods were implemented, comments were added at key points in the code, and patterns were applied. design as MVC and expert to improve the organization and structure of the system.

3.4.2. SCM Plan

Initially, version management was planned using Git and GitHub, with a workflow of work is based on branches (Main, Develop, Feature, and Hotfix). However, due to the Due to the need to speed up development and the lack of experience using Git as a team, the decision was made to by manually sharing the code via WhatsApp, ensuring that both developers always had the latest version of the project.

Despite this change in code management, the established task distribution was respected. in the preliminary draft:

Developer Santiago Perera focused on implementing the backend using Java and Spring Boot, and in the frontend of the internal system's desktop application.

Developer Ulises Castello focused on developing the real estate company's website. with HTML, CSS, React, JavaScript, and Tailwind CSS. It also made improvements to the graphical interface of the desktop application.

This organization allowed each developer to work in their respective area without generating conflicts in implementation.

Although the initial plan was not fully implemented, coordination was maintained.
effective in the development of the system.

3.4.3. Training Plan

Development Team Training:

As part of the preparation for the project, training in new skills was planned.
technologies, as well as the reinforcement of previously acquired knowledge during the
race. Each sprint included a specific time dedicated to training and updates in
the following technologies:

Technologies Learned:

The following technologies were studied through YouTube courses, as well as
through the official documentation of each tool:

React: Development of dynamic interfaces and reusable components.

Spring Boot: Creating REST APIs and backend development in Java.

Java: Code optimization, design patterns, and best practices.

HTML and CSS: Structure and design of web interfaces.

Tailwind CSS: Fast and efficient styling of interfaces using utility classes.

JavaScript: Client-side programming and web interactivity development.

Learning Strategy:

-Weekly individual and team study sessions were established.

-Proof-of-concept tests and small parallel projects were carried out to solidify knowledge.

-The lessons learned were directly applied to the development of the system during each sprint.

-Continuous learning was promoted through the consultation of official documentation.

Customer Training:

To ensure the correct use of the system by the client, sessions were implemented. in-person training:

-Practical sessions were scheduled with clients to demonstrate the use of the system and its functionalities.

-Direct interaction with the system was allowed, resolving doubts in real time.

-At the end of each sprint, customer feedback was collected to ensure understanding progressive and contextual of the system.

4. Conclusions

4.1. Achievement of objectives

Throughout the project's development, the main objectives were achieved, significantly improving the real estate agency's management and optimizing its operations. First, a website was created that allows for the organized and attractive display of properties for rent or sale. In addition, a contact form was incorporated to schedule visits and inquire about properties, which enhances the agency's image and facilitates interaction with potential clients.

Customer and rental information was also digitized, eliminating the need to manage physical records and improving data access through a centralized platform. This allows for faster and more accurate information retrieval, optimizing the real estate agency's daily operations and minimizing management errors.

To improve operational management, an alert system was implemented that automatically notifies of contract expirations, outstanding payments, and missing name changes for property services. This prevents delays and improves planning, reducing potential issues for clients and property owners.

Finally, a module for generating settlement reports and issuing detailed rent receipts was implemented. This not only provides a clear and organized view of income but also improves transparency and financial control for landlords.

Beyond the immediate benefits, these improvements have laid the foundation for the long-term evolution and growth of the business. The digitization of information and the automation of processes have optimized staff time, reduced operating costs, and improved the customer experience. Furthermore, the centralization of data facilitates strategic decision-making based on accurate information, allowing the real estate company to identify market trends, optimize its property portfolio, and improve financial planning.

The system has also been designed with the flexibility to adapt to future needs. Its architecture allows for potential expansion, such as integration with digital marketing tools, the incorporation of new functionalities based on user feedback, or even the automation of other administrative processes. In this way, the real estate agency has not only modernized its current management but is also prepared to face the challenges of the sector with greater efficiency and competitiveness.

Furthermore, these improvements have had a direct impact on customer satisfaction, offering faster response times and a more professional and transparent experience. This has helped build customer trust and strengthen the real estate agency's image in the market.

Another key aspect has been information security. Digitization has reduced the risks associated with the loss or deterioration of physical documents, ensuring more secure and accessible storage. Implementing measures to protect data privacy also strengthens the trust of both customers and owners.

Finally, the real estate company has differentiated itself from its competitors by adopting advanced technological solutions. Process optimization and improved property management give it a competitive edge in a constantly evolving sector, positioning it as an innovative company prepared for the challenges of the future.

4.2. Final analysis of the overall effort

In the first timeline we defined in the preliminary project, we estimated that we were going to dedicate 817 total hours to project tasks, which are subdivided into tasks of Implementation, Meetings, Training, Analysis, Testing and Other (hours used of contingency buffer for each sprint).

We ultimately dedicated a total of 713 hours to the project, which were divided as follows: as follows:

- **Implementation:** 300.5 hours.
- **Meetings:** 56 hours.
- **Training:** 124 hours.
- **Analysis:** 124 hours.
- • **Testing:** 32.5 hours.
- **Other:** 76 hours.

Chart 1. Hours by task type

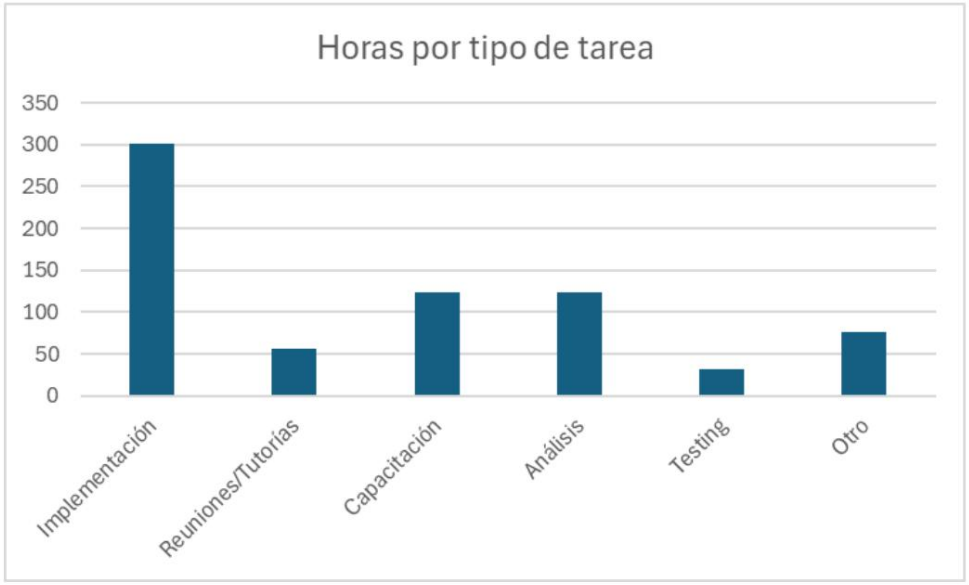


Chart 2. Estimated hours vs. Actual hours



4.3. Customer satisfaction

To assess customer satisfaction, they were asked to complete a survey.

The platform consisted of the following 2 questions:

At what level do you feel the product met your expectations?



How would you describe the communication with the developers?



4.4. Lessons learned

During the development of the project, we identified positive aspects that we would like to repeat. in future developments, as well as areas for improvement to optimize our work in the future opportunities.

4 things we did well and would do again:

- **Regular follow-up meetings:** Constant communication between the developers were able to detect problems early and make timely decisions.
- **Continuous communication with the client:** We maintained fluid communication and constant communication with the client to ensure their needs and expectations are met. were aligned with the project's development. This constant feedback This allowed us to make timely adjustments and deliver a product that truly satisfied. their requirements.
- **Clear and structured technical documentation:** Detailed documentation was generated of the requirements, architecture and development of the project, facilitating the continuity of work and future improvements.
- **Teamwork and collaboration:** Smooth cooperation between developers It allowed for quick problem-solving and improved code quality. This A collaborative approach was key to the project's success.

4 things we would do differently if we were to do it again:

- **More detailed planning of development times:** Some tasks took longer than expected, so in future projects we will be more precise in the estimation of times and margins of error.
- **Clearer definition of requirements from the outset:** In some cases, the Requirements changed as development progressed, which involved adjustments not planned. A more thorough analysis phase would have prevented this.
- **Test automation:** The lack of automated testing led to the implementation of Many manual checks. In future projects we will implement testing tools to improve efficiency and reliability.

- **Greater focus on scalability from the outset:** In the current project, some Design decisions did not fully consider long-term scalability, which could create difficulties when adding new features or when managing a larger volume of data. In future projects, we will pay more attention to designing the architecture in a way that is easily scalable from the beginning, to avoid costly restructurings in later stages.

4.5. Product improvements

Improvements that could be implemented in the internal system:

Data Export: Allow exporting customer, property, and contract listings to CSV, Excel or PDF formats.

Digital signing of contracts: Integrate electronic signatures for rental agreements and official documents.

In-system alerts: Notify within the system itself on a dashboard notifications of important events (due dates, late payments, etc.).

Tenant rating system: Allows managers to rate tenants based on their payment history and behavior.

Rent readjustment automation: Automatically calculate annual increases according to inflation indices or contract.

Edit properties: It will be necessary to implement the option to edit properties for that the customer can update the names of the UTE and OSE account holders. Even though not Even though it is not yet implemented, the use case for this requirement is developed in the Annex 1 (CU015).

Improvements that could be implemented on the website:

User registration and custom accounts: Allowing owner clients and Tenants need an account to view relevant information.

Interactive property map: Display properties on a map for easier access search by location.

Comments and ratings system: Allow tenants to leave reviews about properties and experience with the real estate agency.

Social media integration: Automatically publish new properties on social networks like Instagram and Facebook.

Glossary

Java: Programming language used for the development of the real estate management system.

Spring Boot: A Java framework that facilitates the rapid development of robust and scalable applications.

HTML, CSS, JavaScript, and React: Web technologies used to develop the real estate website. HTML defines the structure, CSS the visual style, JavaScript enables interactivity, and React facilitates the construction of dynamic user interfaces.

Personal Income Tax (IRPF): This tax is levied on income derived from renting properties. Some landlords may be exempt depending on the rental amount.

UTE and OSE services: Electricity and water services in Uruguay that must be paid for by tenants and whose arrears are monitored by the system.

MySQL: Relational database management system used to store system information.

Tailwind CSS: A CSS framework that simplifies website design and styling through predefined classes.

AWS (Amazon Web Services): Cloud platform used for system deployment and data storage.

Elastic Beanstalk: An AWS service that enables the automated deployment and management of cloud applications.

RDS (Relational Database Service): AWS cloud database service used to host the system database.

UX (User Experience): Practices focused on improving the usability and accessibility of the system.

UI (User Interface): Visual design and graphical components with which users interact in the application.

Git/GitHub: Version control system used for source code management and collaboration between developers.

Deploy: The process of publishing and putting an application into operation in a production or testing environment.

Literature

[1] Asana, “Scrum: Key Concepts and How It Is Applied in Project Management”, February 2024, [Online]. Available: https://asana.com/es/resources/what-is-scrum

[2] Meta, “React Documentation”, 2024, [Online]. Available: https://react.dev/

[3] Spring, “Spring Boot Reference Documentation”, 2024, [Online]. Available: https://docs.spring.io/spring-boot/docs/current/reference/html/

[4] Tailwind Labs, “Tailwind CSS Documentation”, 2024, [Online]. Available: https://tailwindcss.com/docs

Annexes

Annex 1. Development of use cases

Testing Use Case: User Login

ID: CU001

Use Case Name: User Login

Objective: To verify that the system allows users to authenticate correctly with

Validate credentials and reject authentication attempts with invalid credentials.

Actors

User: Person attempting to log into the system.

Authentication System: Component that validates credentials and manages access.

Preconditions

- The authentication system is active.
- The user has valid credentials (username and password).

• The database contains the user records required for the test.

Postconditions

• The authenticated user is redirected to the customer registration window.

In case of error, an appropriate message is displayed to the user.

Use Case: Customer Registration

ID: CU002

Use Case Name: Customer Registration

Objective: To allow an administrator to register new customers in the system, validating the data.

Actors:

• Administrator: Person who enters the data of the new client.

• Management System: System that stores customer information.

Preconditions:

• The administrator is logged in and has permission to register new customers.

• The customer registration page is accessible.

Postconditions:

• The client is registered in the database with all their data.

• The system displays a confirmation message.

• In case of error, the system displays an error message with the corresponding details.

Main Flow:

-The Administrator accesses the customer registry in the system.

-The system displays a form to complete the customer's information.

-The Administrator enters the required customer data:

- Name
- Last name
- Address
- Identity card

-The Administrator clicks on “Register” to register the client.

-The system validates that all required fields are complete and that the data is in the correct format (fields are not empty).

-The system checks if the identity card is already registered.

- If the ID card is already registered, the system asks the Administrator if they wish to edit the data. existing customer.
- If the Administrator accepts, the system allows editing of the data of the already registered customer.

-If the Administrator rejects, the system cancels the registration.

-If the ID card is not registered or the Administrator chooses to register a new client, the system saves the data in the database.

-The system displays a success message confirming the customer's registration.

Alternative Flow:

-FA-1: Incomplete data

• If one or more required fields are empty, the System displays an error message indicating the specific fields that need to be corrected.

• The Administrator corrects the data and attempts registration again.

-FA-2: ID card already registered

• If the entered identity card number is already registered in the system, the system asks the Administrator if they wish to edit the existing customer data.

• If the Administrator chooses to edit, the system allows modification of the registered customer's data.

• If the Administrator chooses not to edit, the system cancels the registration and displays an informational message.

Use Case: Viewing customer record (partial)

ID: CU003

Use Case Name: Viewing Customer Record (Partial)

Actors:

- **User (Administrator)**

Objective: To allow the user to view partial information about a client in the system.

Preconditions:

- The client and its properties must be defined in the system code (hardcoded).

Postconditions:

- Basic customer information is displayed on screen.

Main Flow:

- The user accesses the customer display functionality in the system.
- The system retrieves the selected customer's data from the code. • The system displays a partial customer record that includes:

-Name

-Last name

-Address

-Property list

Use Case: Add property to an owning client

ID: CU004

Use Case Name: Add property to an owning client

Actors:
User (Administrator)

Objective: To allow the user to register a new property in the system.

Preconditions:

-The administrator must be logged into the system (RF1).
-The client owner to whom the property will be associated must be registered in the system (RF2).

Main flow

-The use case begins when the administrator selects an owner client in the interface.
-The system displays a form for property registration.
-The administrator enters the required information: Address, Description, Number of bathrooms, Number of rooms, Number of bedrooms, Number of parking spaces, Neighborhood, City, Number of floors, Allows pets, Type, Total square meters, Built square meters, Land square meters, Current UTE holder, Current OSE holder, Roof type and Photos.

-The administrator confirms the property registration.
-The system validates the entered data.
-If the data is valid, the system stores the property by associating it with the selected owner client.

-The system notifies the administrator that the registration was successful.

Alternative Flow

- FA-1: Incomplete data

-If one or more required fields are empty, the System displays an error message indicating the specific fields that need to be corrected.
-The Administrator corrects the data and tries the registration again.

Postconditions:

-The property is registered in the system, associated with the corresponding owner client.
-The property can be viewed in the client's record.

Use Case: Deleting a customer's property

ID: CU005

Use Case Name: Deleting a customer's property

Actors:

• **User (Administrator)**

Objective: To allow the user to delete a property registered in the system.

Preconditions:

• The administrator must be logged into the system (RF1).
• The property must be registered and visible in the client's property listing.

Main Flow

• The administrator accesses the property management module. •
Selects a property from the list of properties associated with a client.
• The system displays the details of the selected property. • The
administrator confirms the deletion of the property.
• The system validates the request, removes the property-related data, and updates
the database.
• The system notifies the administrator that the property was successfully deleted.

Alternative flow

FA-1: The administrator attempts to delete without selecting a property in the list.

• The administrator presses the "Delete" button without having selected one
property.

• The system detects that no property has been selected. • The system
displays a warning message: "You must select a property before
of trying to eliminate it."
• The system remains in the property list, waiting for a selection.
property.

Postconditions

• The deleted property should not appear in the property list.
• The owner's customer information must be kept complete and up-to-date.

Use Case: List registered contracts

ID: CU006

Use Case Name: List registered contracts

Actors:

• **User (Administrator)**

Objective: To allow the user to view the list of contracts registered in the system for a specific client.

Preconditions:

• The administrator must be logged into the system (RF1).
• A customer must have been selected from the customer list and the button pressed "View details".

Main Flow

• The administrator selects a client from the list.
• Press the "View details" button.
• The system displays customer data and lists the customer's registered contracts.

Use Case: Add property to an owning client (updated attributes)

ID: CU007

Use Case Name: Add property to an owning client (updated attributes)

Actors:

• **User (Administrator)**

Objective: To allow the user to register a new property in the system.

Preconditions:

• The administrator must be logged into the system (RF1). • The owner client to whom the property will be associated must be registered in the system (RF2).

Main flow

• The use case begins when the administrator selects a customer owner in the interface.

• The system displays a form for property registration.

• The administrator enters the required information: Address, Description, Quantity Number of bathrooms, Number of rooms, Number of parking spaces, Number of bedrooms Neighborhood, City, Number of floors, Pets allowed, Property type Total square meters, Built square meters, Land square meters, Current UTE holder, current OSE holder, roof type and photos.

• The administrator confirms the property registration. • The system validates the entered data.

• If the data is valid, the system stores the property, associating it with the client.
selected owner.

• The system notifies the administrator that the registration was successful.

Alternative Flow

• **FA-1:** Incomplete data

If one or more required fields are empty, the System displays a message error indicating the specific field that needs to be corrected.

The Administrator corrects the data and tries to register again.

Postconditions:

•

• The property is registered in the system, associated with the owning client.
corresponding.

• The property can be viewed in the customer's record.

Use Case: Issuance of rental receipts:

ID: CU008

Use Case Name: Issuance of rental receipts

Actors:

• **User (Administrator)**

Objective: To allow the user to issue a payment receipt for a selected rental.

Preconditions:

• The administrator must be logged into the system (RF1).

Main flow

- The use case begins when the administrator selects a rental in the rental listings of a specific landlord client.
- The system has a combo box that allows you to select the payment month that It will be shown on the receipt.
- The administrator selects a month in the combo box and presses the button "Generate receipt".
- A File Chooser opens in which the administrator selects the name of the PDF file that will be generated and the location where it will be saved.
- Pressing "Save" saves the PDF file and displays the message "Receipt successfully generated."

Postconditions:

- The PDF file of the rent receipt is successfully generated in the location specified by the administrator.

Use Case: Issuance of monthly rent settlement report

ID: CU009

Use Case Name: Issuance of monthly rent settlement report

Actors:

- **User (Administrator)**

•

Objective: To allow the user to issue a monthly rent settlement report.

Preconditions:

• The administrator must be logged into the system (RF1).

Main flow

• The use case begins when the administrator selects a rental in the rental listings of a specific landlord client.

• The system has a combo box that allows you to select the payment month that will be included in the report.

• The administrator selects a month in the combo box and presses the button "Generate monthly settlement report."

A window opens showing the initial rental amount and the various deductions that are made to said value if applicable (VAT, Personal Income Tax and Commission) and it is allowed to add extra expenses indicating the amount and concept. Once the user finishes adding extra expenses, they press the "Generate" button.

• A File Chooser opens in which the administrator selects the name of the PDF file that will be generated and the location where it will be saved.

• Pressing "Save" saves the PDF file and displays the message "PDF generated successfully."

Alternative flow:

FA-1: When trying to add an extra expense, the "Concept" field or the "Amount" field is left empty.

Postconditions:

- The monthly settlement report PDF file is successfully generated in the location specified by the administrator.

Use Case: Customer's Property Stock

ID: CU010

Use Case Name: Customer Property Stock

Actors:

- User (Administrator)

Objective: To allow the administrator to view the number of free properties (without associated rentals) and occupied by a client when opening their profile.

Preconditions:

- The administrator must be logged into the system (RF1).

- The administrator must have selected a client from the client list and opened your file.

Main Flow

1. The administrator selects a customer's record from the customer list in the system interface.

2. The system displays the customer's record.

3. The system calculates:

Vacant properties: Subtracting the number of occupied properties (for rent) from the total number of client properties.

Occupied properties: Equal to the number of rentals associated with the properties of customer.

4. Values are displayed in real time using labels on the customer record.

Alternative flow

4A. Client with no registered properties

-If the client has no registered properties, the system displays "0" in the values for free and occupied properties.

4B. Modified properties (add/remove)

-If a customer property is added or removed, or a rental is added or removed, the system
It automatically updates the values of vacant and occupied properties using a
callback.

-The administrator can view the updated values without needing to refresh.
manually fill out the customer's record.

Postconditions

-The customer's property stock is updated and correctly reflected in the interface.

-The data is consistent and synchronized with the add or delete operations
properties or rentals.

Use Case: Notifications of late payments

ID: CU011

Use Case Name: Payment Delay Notifications

Actors:

• User (Administrator)

Objective: To automate the process of identifying and notifying tenants with payments

Overdue payments can be made by sending an email to the real estate agency with the relevant information. This

It allows the real estate agency secretary to receive an updated list of tenants every

Monday at 10 AM, optimizing the monitoring and management of collections without the need for

perform manual checks on the system.

Preconditions:

-The system must be operational and have internet access.

-There must be at least one rental with overdue payments registered in the system.

-The real estate agency's email address must be correctly configured in the system.

Main Flow

-Every Monday at 10 AM, the system automatically executes the method responsible for sending notifications of delays.

-The system scans the database for rentals with overdue payments.

-For each overdue rent, the system obtains:

1. Tenant's name.
2. Address of the rented property.
3. Tenant's telephone number.
4. Number of months of payment delay.

The system generates an email for each tenant with the collected information.

The system uses JavaMailSender to send emails to the real estate agency's address.

-The secretary receives the email in the inbox.

-The secretary checks the mail and contacts tenants who are behind on their payments.

-Once the call has been made, the secretary marks the email as read.

Alternative Flow

7A. There are no tenants with overdue payments

-The system runs the process and finds that there are no overdue payments.

-No email is sent.

Postconditions

-The real estate agency receives an email for each tenant in arrears.

-The secretary takes action by contacting the tenants.

Use Case: Expiration Notifications

ID: CU012

Use Case Name: Expiration Notifications

Actors:

• User (Administrator)

Objective: To automate the process of identifying and notifying deadlines for rent, by sending an email to the real estate agency with the relevant information.

This allows the real estate agency secretary to receive an email every day at 9 AM.
every contract that expires on that day.

Preconditions:

-The system must be operational and have internet access.

-There must be at least one rental agreement that expires on the date shown.

-The real estate agency's email address must be correctly configured in the system.

Main Flow

-Every day at 9 AM, the system automatically executes the method responsible for sending
expiry notices.

-The system scans the database for contracts that expire on the current date.

-For each contract expiring on the current date, the system obtains:

1. Tenant's name.
2. Address of the rented property.
3. Name of the property owner.

-The system generates an email for each contract with the collected information.

-The system uses JavaMailSender to send emails to the real estate agency's address.

The secretary receives the email in her inbox and is made aware of the situation.

tenant's contractual agreement.

Alternative Flow

7A. There are no contracts expiring on the current date

-The system runs the process and finds that there are no contracts expiring on the current date.

-No email is sent.

Postconditions

-The real estate agency receives an email for each contract that expires on the current date.

Use Case: Property Finder

ID: CU013

Use Case Name: Property Finder **Actors:**

- **Target** User (Client) :

Allow real estate clients to search for properties on the website by applying filters.
according to their needs, such as neighborhood, type of property, number of rooms, number of garages and whether pets are allowed.

Preconditions:

- The website must be accessible from the internet.
- There must be at least one property registered in the database.

Main Flow:

1. The user accesses the real estate website.
2. The user selects the "Property Search" option.
3. The system displays the available filters:
    a. Neighborhood
    b. Type of property
    c. Number of rooms
    d. Number of garages
    e. Accepts pets (Yes/No)
4. The user enters the search criteria and presses the "Search" button.
5. The system queries the database and filters the properties that match with the criteria entered.
6. The system displays the list of properties that meet the filters. applied.

Alternative Flow:

5A. There are no properties that match the selected filters:

-The empty property catalog is displayed.

Postconditions:

- The user views the list of properties filtered according to the criteria admitted.

Use Case: Property Inquiry

ID: CU014

Use Case Name: Property Inquiry

Actors:

-User (Client)

Objective: -To allow website users to contact the real estate agency through a form, sending their inquiry via email or WhatsApp.

Preconditions:

-The website must be accessible from the internet.

-The user must complete the form with at least one message and select a method of contact.

Main Flow:

-The user accesses the real estate agency's website.

-Scroll to the bottom of the homepage, where you will find the form for consultation.

-The user enters the following information in the form:

- Full name (required)
- Message with your query (required) • Email address (required if you choose the email contact option)
- Contact option (WhatsApp or email)

-If the user selects WhatsApp, the system generates a link with the pre-filled message and redirects the user to the WhatsApp application.

-If the user selects email, the system sends an email with the query to the real estate agency address.

-The user receives an on-screen confirmation indicating that their message has been sent correctly.

Alternative Flow:

4A. The user does not complete all required fields:

-The system displays an error message indicating which fields are missing.

Postconditions:

-If the user chose WhatsApp, they are redirected to the application with the pre-filled message.

-If you chose email, the real estate agency receives an email with the user's inquiry.

Use Case: Edit Property

ID: CU015

Use Case Name: Edit Property

Actors:

- User (Administrator)

Objective: To allow the user to modify the data of a property already registered in the system.

Preconditions:

- The administrator must be logged into the system (RF1).
- The property to be edited must be registered in the system and associated with a client owner (RF3).

Main flow:

- The use case begins when the administrator selects a property in the interface.
- The system displays the current property data in an editable form.
- The administrator modifies the fields they wish: Address, Description, Number of bathrooms, Number of rooms, Number of bedrooms, Number of Garages, Neighborhood, City, Number of floors, Pets allowed, Type, Square meters Total square meters, Built square meters, Land square meters, Owner Current UTE, Current OSE Owner, Roof Type and Photos.
- The administrator confirms the property edit.
- The system validates the entered data.
- If the data is valid, the system updates the property with the new data.
- The system notifies the administrator that the edit was successful.

Alternative Flow:

- FA-1: Incomplete or incorrect data If one or more required fields are

If the fields are empty or contain invalid information, the system displays an error message. indicating the specific fields that need to be corrected. The administrator corrects the data. and try editing again.

Postconditions:

- The property is updated in the system with the modified data.

- Changes can be viewed in the owner customer's record.

Appendix 2. User Manual

Login: https://youtu.be/jX5rmSsW2kg

Main customer management features: https://youtu.be/ZR65MQERC24

Appendix 3. Deployment Manual

Website deployment:

We deployed the real estate website using AWS S3 (Amazon) Simple Storage Service), a cloud storage service that allows To efficiently host static content. These were the steps we followed in the process:

• Application Build:

First, we run a build of the web application. This process involves compiling and optimize all source files (HTML, CSS, JavaScript, images, etc.) in an optimized version ready to be deployed in a production environment.

The result of this build was a dist/ folder, which contains all the static files that the application needs to function properly.

• Creating a bucket in AWS S3:

We created a bucket in AWS S3, which is a logical container for storing the files. Buckets in S3 act as file repositories, and each file
Within the bucket, access can be public or private, depending on the permission settings.
We configured the bucket to host static content, which allowed us to
It will allow serving web files directly from S3.

• Uploading files to the bucket:

We uploaded the dist/ folder to the S3 bucket. This included all generated files. during the build process, such as index.html files, CSS files, scripts JavaScript and other static resources.

• Homepage settings:

Within the bucket, we assign the index.html file as the main page of the application. This was done by configuring index.html as the document default that will be served when users access the site.

• Permission settings:

We adjusted the bucket permissions so that the files are public and accessible.
via the internet. This allowed any user who entered the URL of the
See the website content.

Deployment API backend:

To carry out the deployment of the Spring Boot backend API that we use in
For our project, we followed a series of detailed steps that allowed us to
Deploy the application efficiently and scalably in the cloud using AWS
(Amazon Web Services). Here I explain each part of the process in detail:

- Creating the database in AWS RDS:

AWS RDS (Relational Database Service) is a managed service that facilitates
Creation, administration, and scaling of relational databases. In our case,
We decided to use RDS to manage our backend API database.

We begin by creating an RDS database instance, selecting the type of
suitable database engine (in our case we use MySQL).

During database configuration, we define parameters such as:

-Database name: We assign a unique name to our database.
-Username and password: We set up an administrator user with
secure credentials.
-Security settings: We adjusted the security rules to allow
Our application in Elastic Beanstalk accesses the database.
-Subnet and security group: We made sure the database was located
on a private subnet.
Once the database was ready, we obtained the RDS connection endpoint.
(i.e., the database URL) that we would need to connect our
application with the database.

• Deploying the API in Elastic Beanstalk:

AWS Elastic Beanstalk is a platform as a service (PaaS) that facilitates Application deployment, management, and scalability, such as our Spring API Boot. Elastic Beanstalk handles many of the operational tasks, such as provision servers, load balance, and manage scalability and monitoring.

To deploy the API, we first create an Elastic Beanstalk environment with the following configurations:

We select the Java platform type and the corresponding Spring Boot version. We set the environment parameters (environment name, instance type, etc.). Next, we package our Spring Boot application into a JAR file by running a maven build.

We uploaded the packaged file to Elastic Beanstalk using the AWS console. Elastic Beanstalk then took care of creating an EC2 instance (virtual machine) and deploy our API's JAR file, exposing the API services in a web environment.

Elastic Beanstalk was also responsible for managing load balancing and traffic distribution across instances, which ensured availability and the proper performance of the API.

• Setting up the connection between the API and the RDS database: Once

The database and API were deployed; it was necessary to establish the connection. between our Spring Boot application and the RDS database. To do this, We modified the application's application.properties configuration file. Spring Boot.

In this file, we configure the following properties:

-Database URL: We use the database endpoint that we obtained from the Create the RDS instance. This is set in the spring.datasource.url property.

Credentials: We enter the database username and password in the spring.datasource.username and spring.datasource.password properties.

-JDBC Driver: We configured the JDBC driver as MySql.

Appendix 4. Testing Evidence

CU001:

ID Escenario	Descripción del Escenario	Datos de Entrada	Resultado Esperado
1	Login exitoso con credenciales válidas	Usuario: admin Contraseña: admin	El sistema permite el acceso y redirige a la página principal del usuario.
2	Login fallido con contraseña incorrecta	Usuario: admin Contraseña: 1234	El sistema rechaza el acceso y muestra el mensaje: "Credenciales incorrectas."
3	Login fallido con nombre de usuario no registrado	Usuario: Santiago Contraseña: 1234	El sistema rechaza el acceso y muestra el mensaje: "Credenciales incorrectas".
4	Login fallido con campo de usuario vacío	Usuario: "" Contraseña: 1234	El sistema rechaza el acceso y muestra el mensaje: "El nombre de usuario no puede ser vacío".
5	Login fallido con campo de contraseña vacío	Usuario: Santiago Contraseña: ""	El sistema rechaza el acceso y muestra el mensaje: "La contraseña no puede ser vacía".
6	Login fallido con campos de usuario y contraseña vacíos	Usuario: "" Contraseña: ""	El sistema rechaza el acceso y muestra el mensaje: "El nombre de usuario no puede ser vacío".

Test 1 result: OK

Test 2 result: OK

Test 3 result: OK

Test 4 result: OK Test 5 result: OK Test 6 result: OK

CU002:

ID Escenario	Descripción del Escenario	Datos de Entrada	Resultado Esperado
1	Registro exitoso de cliente con todos los campos completados correctamente.	Nombre: "Ulises" Apellido: "Castello" Dirección: "Colonia 1091"	El cliente se registra correctamente y se muestra el mensaje "Registro exitoso"
2	Intento de registro con nombre vacío.	Nombre: "" Apellido: "Perera" Dirección: "18 de julio 2011"	El sistema muestra el mensaje de error "El nombre del cliente no puede ser vacío"
3	Intento de registro con apellido vacío.	Nombre: "Santiago" Apellido: "" Dirección: "Soriano 1420"	El sistema muestra el mensaje de error "El apellido del cliente no puede ser vacío"
4	Intento de registro con dirección vacía.	Nombre: "Santiago" Apellido: "Perera" Dirección: ""	El sistema muestra el mensaje de error "La dirección del cliente no puede ser vacía"
5	Intento de registro con nombre, apellido y dirección vacías.	Nombre: "" Apellido: "" Dirección: ""	El sistema muestra el mensaje de error "El nombre del cliente no puede ser vacío"
6	Intento de registro con nombre y apellido vacíos.	Nombre: "" Apellido: "" Dirección: "18 de julio 1080"	El sistema muestra el mensaje de error "El nombre del cliente no puede ser vacío"
7	Intento de registro con apellido y dirección vacías.	Nombre: "Ulises" Apellido: "" Dirección: ""	El sistema muestra el mensaje de error "El apellido del cliente no puede ser vacío"
8	Intento de registro con nombre y dirección vacías.	Nombre: "" Apellido: "Castello"	El sistema muestra el mensaje de error "El nombre del

		Dirección: ""	cliente no puede ser vacío"

Test 1 result: OK

Test 2 result: OK

Test 3 result: OK

Test 4 result: OK

Test 5 result: OK

Test 6 result: OK

Test 7 result: OK

Test 8 result: OK

CU003:

ID Escenario	Descripción del Escenario	Resultado Esperado
1	Se accede a la ventana de la visualización de una ficha de cliente.	Se muestran los datos del cliente hardcodeados en el sistema exitosamente.

Test 1 result: OK

CU004:

ID Escenario	Descripción escenario	Datos de Entrada	Resultado Esperado
1	Registro exitoso de propiedad con todos los campos completados correctamente.	Descripción: "Descripción 1" Dirección: "Dirección 1"	El cliente se registra correctamente y se muestra el mensaje "Propiedad registrada con éxito"
2	Intento de registro con dirección vacía.	Descripción: "Descripción 2" Dirección: ""	El sistema muestra el mensaje de error "La dirección de la propiedad no puede ser nula"
3	Intento de registro con descripción vacía.	Descripción: "" Dirección: "Dirección 3"	El sistema muestra el mensaje de error "La descripción de la propiedad no puede ser nula"
4	Intento de registro con descripción y dirección vacía.	Descripción:"" Dirección:""	El sistema muestra el mensaje de error "La dirección de la propiedad no puede ser nula"

Test 1 result: OK

Test 2 result: OK

Test 3 result: OK Result

Test 4: OK

CU005:

ID Escenario	Descripción Escenario	Datos de Entrada	Resultado Esperado
1	El cliente selecciona una propiedad del listado de propiedades, presiona el botón “Eliminar propiedad seleccionada” y confirma la eliminación.		La propiedad se elimina exitosamente y el sistema muestra el mensaje “Propiedad eliminada exitosamente”.
2	El cliente presiona el botón “Eliminar propiedad seleccionada” sin haber seleccionado una propiedad del listado previamente.		El sistema muestra el mensaje “Seleccione una propiedad para eliminar”.

Test 1 result: OK Result

Test 2: OK

CU006:

ID Escenario	Descripción del Escenario	Datos de Entrada	Resultado Esperado
1	Se selecciona un cliente del listado de clientes y se presiona el botón “Ver ficha”.		Además de los datos del cliente y su listado de propiedades, se muestra su listado de contratos registrados.

Test 1 result: OK

CU007:

Scenario ID	**Scenario description**	**Data of entrance**	**Result Expected**
1	Record successful of property with all fields completed correctly.	Description: “Example property” Address: “Address 1” Property type: "Home" Neighborhood: “Charrúa” City: "Juan Lacaze” Number of of rooms: 2 Number of of plants: 1	The property is registered successfully, the message is displayed “Photos successfully captured” and then the message displayed is “Registered property with success”

		Number of bathrooms: 1 of	

		Number of bedrooms: 2 of Total square meters: 100 Square meters built: 50 Square meters of land: 50 UTE Owner: “Juan Perez” Holder OSE: “Juan Pérez” Roof type: "Sheet" Number of parking spaces: 1 Pets allowed: true Photos: A photo with the .jpg extension was uploaded	
2	Property of Registry with field “Description” empty	Description: "" Address: “Direction 2” Property type: "Local commercial" Neighborhood: "Villa Pancha”	The message “Error registering the property: The description can be empty” is No displayed, the property is not registered.

		City: "Juan Lacaze” Number of rooms: 2 Number of bedrooms: 0 Number of plants: 2 Number of bathrooms: 1 Total square meters: 200 Square meters built: 100 Square meters of land: 100 UTE Owner: “Juan Pérez” Holder OSE: “Juan Pérez” Roof type: “Ironed” Number of parking spaces: 1 Accepts pets: false Photos: There are two They uploaded photos with the .jpg extension.	

3	Property of Registry with	Description: “Description 3”	The message “Error to

	field “Address” empty	Address: "" Property type: "Apartment" Neighborhood: "Three Spotlights” City: "Juan Lacaze” Amount of environments: 1 Number of bedrooms: 1 Number of plants: 1 Number of bathrooms: 1 Total square meters: 40 Square meters built: 40 Square meters of land: 0 UTE Owner: “Juan Pérez” Holder OSE: “Juan Pérez” Roof type: “Ironed” Amount of garages: 0	Register the property: The address No can be empty”, the property is not registered.

		Accepts pets: false	

		Photos: There are two They uploaded photos with the .jpg extension.	

4	Property registration of with “Current holder empty UTE”	Description: “Description 4” Address: “Tabobo 32” Property type: "Apartment" Neighborhood: “Charrúa” City: "Juan Lacaze” Number of rooms: 1 Number of bedrooms: 1 Amount of plants: 1 Number of bathrooms: 1 Total square meters: 40 Square meters built: 40 Square meters of land: 0 UTE Owner: “” Holder OSE: “Juan Pérez”	The message “Error registering property: The current holder of "UTE cannot be empty," the property is not registered.

		Roof type: “Ironed” Number of parking spaces: 0 of Accepts pets: false Photos: There are two They uploaded photos with the .jpg extension.	
5	Property registration with “Current holder of OSE” empty of	Description: “Description 5” Address: “Tabobo 32” Property type: "Home" Neighborhood: “Freedom This" City: "Juan Lacaze” Amount of environments: 1 Number of bedrooms: 2 of Number of plants: 1 of Number of bathrooms: 1 of Total square meters: 150	The The message “Error registering the property: The current OSE holder cannot be empty” means the property is not registered.

		Square meters built: 100 Square meters of land: 50 UTE Owner: “Juan Pérez” OSE Holder: “” Roof type: "Sheet" Number of parking spaces: 0 Accepts pets: false Photos: There are two They uploaded photos with the .jpg extension.	

6	Property registration of without upload photos	Description: “Description 6” Address: “Tabobo 32” Property type: "Apartment" Neighborhood: "Island Bad" City: "Juan Lacaze” Number of rooms: of 1 Number of bedrooms: of 1	The message “Error registering the property: You must upload photos for the property” is displayed.

		Number of of plants: 1 Amount of bathrooms: 1 Total square meters: 150 Square meters built: 150 Square meters of land: 0 Holder UTE: “Juan Pérez” Holder OSE: “Juan Pérez” Roof type: "Sheet" Number of of parking spaces: 0 Accepts pets: true Photos: No photos were uploaded	

Test 1 result: OK

Test 2 result: OK

Test 3 result: OK

Test 4 result: OK

Test 5 result: OK

Test 6 result: OK

CU008:

Scenario ID	Description Scenario Input Data Result		Expected
1	A rental is selected from the rental list of a client owner, the month “January” is chosen in the month combo box, the “Generate report” button is pressed and the file is saved to the Desktop with the name “rental_report.pdf”.	Month: “January”	The The message “PDF generated successfully” is displayed and the receipt is generated in the specified location.

Test 1 result: OK

CU009:

Scenario ID	Description Scenery	Data of Entrance	Result Expected
1	Select a rental from a client's rental list, choose the month "June" from the drop-down menu, and click the "Generate Monthly Statement" button. When the window opens, do not add any extra expenses, click the "Generate" button, and save.	Month: “June”	The message “PDF generated successfully” is displayed. and the report is generated in the specified location.

	the PDF file in the Desktop with the name “rental_report.pdf”.		
2	A rental is selected from a client's rental list, the month "May" is chosen from the month dropdown box, and the "Generate Monthly Statement" button is pressed. When the window opens, an extra expense with the description "Plumber" is added, the amount field is left empty, and the "+" button is pressed to add the extra expense to the list.	Month: “May” Concept: "Sanitary" Amount: “”	The error message is displayed “The amount of "Extra expenses cannot be be empty"; the expense is not added to the list.
3	A rental is selected from a client's rental list, the month "December" is chosen from the month dropdown menu, and the "Generate Monthly Statement" button is pressed. When the window opens, an extra expense is added with an empty description field, the amount field is entered with a value of 100, and the "+" button is pressed to add the extra expense to the list.	Month: "December" Concept: "" Amount: 100	The error message “The concept of the extra expense cannot be empty” is be displayed, the expense is not added to the list.

Test 1 result: OK

Test 2 result: OK

Test 3 result: OK

CU010:

Scenario ID	**Description Scenery**	**Input Data Result**	**Expected**
1	Client No properties registered	Client with ID = 1, no associated properties.	Label values: Free properties = 0, Occupied properties = 0.
2	Customer with free properties	Client with ID = 2, 5 registered properties, 0 associated rentals. to	Label values: Free properties = 5, Occupied properties = 0.
3	Client with properties occupied	Client with ID = 3, 4 registered properties, 4 associated rentals. to	Label values: Free properties = 0, Occupied properties = 4.
4	Client with properties vacant and occupied	Client with ID = 4, 10 registered properties, 6 associated to rentals.	Label values: Free properties = 4, Occupied properties = 6.
5	Add a new property to a client with available properties	Client with ID = 5, Initial values: 3 registered properties (3 vacant, 0 occupied). New property added: 1 vacant.	Updated: Free properties = 4 properties, occupied = 0.

6	Add occupied property to client with vacant properties	Client with ID = 6, Initial values: 2 updated: properties Free properties registered (2 = 2, Free properties, 0 occupied). occupied = 1. New property added: 1 occupied (associated with a to rental).	
7	Remove free property	Client with ID = 7, Initial values: 5 Updated: properties Free properties registered 3 = 2, Free properties, 2 occupied). occupied = 2. Property removed: 1 free.	
8	Remove occupied property	Client with ID = 8, initial: 4 properties registered (1 vacant, 1 occupied). 3 Property removed: occupied. 1	The message “The property has an assigned rent, you can remove it” is displayed. Values I don't know They don't change.
9	Automatic stock update	Client with ID = 9, initial: 6 of properties (4 free, 2 occupied). One is removed free property, a new one is added, and This new one is assigned a rental.	Automatically updated values: Free properties = 3, Occupied properties = 3.

Test 1 result: OK

Test 2 result: OK

Test 3 result: OK

Test 4 result: OK

Test 5 result: OK

Test 6 result: OK

Test 7 result: OK

Test 8 result: OK

Test 9 result: OK

CU011:

Scenario ID	**Description Scenery**	**Input Data Result**	**Expected**
1	The system has a registered a tenant with all payments up to date.	Tenant with 0 months of arrears in rent payments.	No notification email is sent.
2	He system has a tenant registered with 1 month arrears in the payments.	Tenant is 1 month behind on rent payments.	An email is sent to the real estate agency with the tenant's full name, phone number, and the address of the rented property, indicating that they owe 1 month's rent.
3	The system has registered a tenant with more than 1 month in arrears.	Tenant with 3 months of arrears in rent payments.	An email is sent to the real estate agency with the tenant's full name, phone number, and the address of the rented property, indicating that they owe 3 months' rent.

4	The system has multiple tenants with overdue payments.	3 tenants with arrears of 2, 3 and 4 months respectively.	An email is sent for each tenant with their respective full names, telephone numbers, addresses of of the rented properties, indicating respective amounts their of months of delay in the payment of rent.

Test 1 result: OK

Test 2 result: OK

Test 3 result: OK

Test 4 result: OK

CU012:

Scenario ID	**Description Scenery**	**Input Data Result**	**Expected**
1	The system has registered a rental agreement that does not expire on the current date.	Contract that does not expire on the date current.	No email is sent from notification.
2	He The system has a registered rental agreement that expires on the current date.	Contract that expires on the current date.	An email is sent to the real estate agency with the tenant's full name, the address of the rented property, and the owner's name.

			property indicating that the rental agreement expires on the current date.
3	The system has multiple rental agreements that expire on the date current.	4 contracts that expire on the current date	An email is sent for each contract with its respective details (name of the complete tenant, address of the rented property and property owner) indicating that they expire on the current date.

Test 1 result: OK

Test 2 result: OK

Test 3 result: OK

CU013:

Scenario ID	**Description Scenery**	**Input Data Result**	**Expected**
1	The user searches for properties without applying filters.	No filters	All properties available in the database are displayed.
2	The user applies a neighborhood filter to find properties in a specific area.	Neighborhood: "Center"	Only the properties located in the neighborhood "Center".

3	The user applies multiple filters to obtain more accurate results.	Neighborhood: "Charrúa", Type: "House", Number of rooms: 3, Pets allowed: Yes	Only the houses in the "Charrúa" neighborhood with 3 rooms that accept pets are shown.
4	The user applies filters that do not match with no property.	Type: "House", Quantity of Parking spaces: 3, Pets allowed: No	The empty property catalog

Test 1 result: OK

Test 2 result: OK

Test 3 result: OK

Test 4 result: OK

CU014:

Scenario ID	**Description Scenery**	**Input Data**	**Result Expected**
1	The user completes the form correctly and choose contact via WhatsApp.	Name: "Juan Pérez", Message: "I'm interested in a house.", Send to: WhatsApp	The user is redirected to WhatsApp with the pre-filled message.
2	The user completes the form correctly and chooses to be contacted by email.	Name: "María López", Message: "I would like information about houses for rent.", Email: "maria@gmail.com", Send to: Gmail	An email is sent to the real estate agency with the inquiry.
3	The user does not enter want your name and more	the message: "I information." Send to: WhatsApp	

	Try sending the query.		
4	The user selects email, but does not enter their email address.	Name: "Carlos Gómez", Message: "I am interested in buying a house.", Send to: Email	The system displays an error indicating that the email is required.

Test 1 result: OK

Test 2 result: OK

Test 3 result: OK

Test 4 result: OK